



Master's Degree Program

Field of Study: Advanced Analysis – Big Data

Thi Van Trang Le

Number ID : 140036

Process Intelligence with Python Language

Master's Thesis

Under the supervision of

dr hab. Michała Ramsza

Institute of Mathematical Economics

Warszawa 2026

Contents

1	Introduction	5
1.1	Background and motivation	5
1.2	Research Objectives and Research Question	6
1.3	Relevance and Academic Context	6
1.4	Methodology	7
1.5	Structure of the thesis	8
2	The mathematics of process mining	10
2.1	Event Logs and Traces	10
2.2	Activities, variants, and directly-follows relations	11
2.3	Process Discovery and Process Models	13
2.3.1	The Alpha algorithm	14
2.3.2	The Inductive Miner and Process Trees	14
2.3.3	Petri Nets	15
2.4	Conformance Checking	16
2.5	Performance Analysis and Process Intelligence	18
2.6	Event Data Representation in Python	19
2.7	Exploratory Description of the BPI Challenge 2019 Dataset	20
3	The Tools Available in Python	22
3.1	Core Process Mining Libraries	22
3.1.1	PM4Py	22
3.1.2	PM4Py-Streaming	23
3.1.3	Simod	24
3.1.4	PMLab	24
3.1.5	BupaR (Python Interface)	25
3.2	Predictive and AI-Oriented Tools	25
3.2.1	ML4ProM	25
3.2.2	PM4PYML	26
3.2.3	PyCelonis	26
3.2.4	BPMN-Python and Workflow Automation	27
3.2.5	Predictive Process Monitoring Toolkit (PPMT)	27
3.3	Simulation and Optimization Tools	28
3.3.1	SimPy	28
3.3.2	BPSim	28
3.3.3	Process Optimizer	29
3.4	Visualization Libraries	29
3.4.1	Graphviz	29
3.4.2	Plotly and Dash	30
3.4.3	NetworkX	30
3.4.4	Pyvis	30
3.5	Complementary Data Processing Libraries	31
3.5.1	Pandas	31

3.5.2	NumPy	31
3.5.3	Scikit-Learn	31
3.5.4	XGBoost and LightGBM	32
3.5.5	TensorFlow and PyTorch	32
3.6	Academic and Community Resources	32
3.7	Summary	33
4	Worked Examples	35
4.1	Introductory Synthetic Example:	35
4.2	BPI Challenge 2019: The Case Study	42
4.2.1	Data Source, Format, and Structure	42
4.2.2	Purpose and Analytical Workflow	44
4.2.3	Implementation and Results	46
4.2.4	Discussion of Results	49
4.3	Advanced Example: Loops, Variants, and Performance Analysis	51
4.3.1	Variant Analysis	51
4.3.2	Loop and Rework Detection	53
4.3.3	Throughput Time and Performance KPIs	54
4.3.4	Implementation Overview	55
4.4	Performance DFG, Automation Rate, and Object-Centric Analysis	60
4.4.1	Performance-Annotated Directly-Follows Graph	60
4.4.2	Automation Rate Analysis	63
4.4.3	Object-Centric Directly-Follows Graph	64
4.5	Simulation and Predictive Monitoring	67
4.5.1	Discrete-Event Simulation with SimPy	67
4.5.2	Predictive Process Monitoring with Random Forest	69
5	Conclusions	72
5.1	Summary of Findings	72
5.2	Limitations	72
5.3	Directions for Future Research	73
	Bibliography	74
	List of figures	75
	List of tables	76

Chapter 1

Introduction

1.1 Background and motivation

These days, almost every business process happens inside digital systems such as ERP platforms, CRM tools, or workflow apps. What's interesting is that every action, whether it's an approval, a payment, or just a status update, gets recorded automatically: who did it, when, and for which case. On their own, these records might not seem important. But when you look at them all together, they reveal how work really happens in practice, which is often quite different from the way the process was originally designed.

When we analyze this data, we gain a much clearer and more honest view of how processes operate day-to-day. Unlike traditional management reports, which often reflect how things are supposed to work, process mining allows us to trace the actual sequence of events by following the digital footprints left in the system. This makes it possible to see where delays or repeated work really happen, and to spot differences between teams, locations, or even individual cases. In many situations, these insights reveal surprising gaps between the process as designed and how it unfolds in reality.

Since the late 1990s, when van der Aalst and his colleagues introduced the first process mining algorithms, the field has developed rapidly. What began as a research topic has now found its way into real businesses around the world. There are dedicated conferences, an official IEEE task force, and a growing number of practical applications in areas such as healthcare, finance, logistics, and manufacturing. This growing interest highlights just how valuable process mining has become, both in theory and in everyday business practice.

Process intelligence builds on the foundation of process mining and takes it a step further. While process mining mainly looks back to analyze what has already happened, process intelligence brings a forward-looking perspective. It involves monitoring ongoing cases in real time, predicting which ones might encounter problems, and even simulating how things could play out if resources were allocated differently. The aim is not only to understand past performance, but also to use that understanding to make better decisions and take action as events unfold.

Python has emerged as the preferred language for process mining and process intelligence work. Its strong presence in the data science community, along with a growing selection of specialized libraries, makes it a natural choice for practitioners. For example, PM4Py is one of the most comprehensive libraries available, covering everything from loading event logs to discovering process models, checking conformance, and analyzing performance. Other libraries extend these capabilities further by offering tools for simulation, prediction, and visualization. As a result, it is now possible to carry out an entire process intelligence project entirely within the Python ecosystem.

Despite these advances, navigating the Python process mining ecosystem can still be challenging. Many libraries offer similar features, and the quality of documentation varies widely. There is also a lack of a single, comprehensive resource to help users understand which tool is best suited for each task, or how to use them effectively in real-world situations. With this thesis, I aim to fill that gap by providing a structured overview of the main tools available, along with practical code examples that readers can try out and adapt to their own needs.

1.2 Research Objectives and Research Question

The main question guiding this thesis is:

Which Python tools are available for process mining and process intelligence, and how well do they support key analytical tasks like process discovery, conformance checking, and performance analysis?

To answer this question, the thesis sets out to achieve four specific objectives:

1. To introduce and explain the mathematical concepts that form the foundation of process mining, such as event logs, traces, directly-follows relations, process models, and conformance measures, so that it's possible to compare different tools in a rigorous way.
2. To review the main Python libraries available for process mining and process intelligence, grouping them by their primary functions, and describing their features, strengths, and limitations in an organized manner.
3. To show how selected Python tools can be used in real-world scenarios, using worked examples that range from simple synthetic datasets to large procurement event logs, and extending beyond basic process discovery to include performance measurement, simulation, and outcome prediction.
4. To draw conclusions about the current state of the Python process mining ecosystem, highlighting any gaps and suggesting directions for future development.

This thesis focuses on Python-based tools and the three core process mining tasks discovery, conformance checking, and performance analysis as well as related capabilities such as simulation and predictive monitoring. The emphasis is on open-source solutions that can be accessed by both practitioners and researchers without the need for commercial licenses. The only exception is PyCelonis, which is included because it represents an important category of commercial process intelligence platforms that offer a Python interface.

1.3 Relevance and Academic Context

Process mining emerged as a research field in the late 1990s and early 2000s, largely thanks to the pioneering work of Wil van der Aalst and his colleagues at Eindhoven University of Technology. Since then, the field has grown significantly, with its own annual conference—the International Conference on Process Mining (ICPM)—and a formal IEEE task force that has established a clear set of principles and goals for process

mining. The textbook by van der Aalst (Aalst, 2022) remains the main reference point for both theoretical and practical aspects of process mining, and it serves as the primary academic resource for this thesis.

One important driver of progress in the field has been the BPI Challenge, which is held each year alongside ICPM. The challenge provides researchers with real-world datasets from organizations and invites them to apply process mining techniques to analyze them. These datasets have become standard benchmarks and are widely used to compare algorithms and tools. For this thesis, I use the BPI Challenge 2019 dataset as the main case study. This dataset, drawn from a large multinational company’s procurement process, includes nearly 1.6 million events and is one of the most complex in the series. Its widespread use in academic studies makes it a strong foundation for demonstrating process mining methods in a realistic context.

The Python ecosystem for process mining has evolved rapidly since the introduction of PM4Py in 2018. Earlier implementations, such as ProM, were Java-based and still see heavy use in academic settings, but they require more specialized expertise to extend and deploy. The arrival of Python-based libraries has made it much easier for both researchers and practitioners to get started, and has allowed process mining to integrate more naturally with other data science tools for machine learning, visualization, and statistical analysis. New libraries have continued to appear, supporting tasks like streaming process mining, simulation, and predictive monitoring. Even so, the Python ecosystem is still less mature and less thoroughly documented than its Java-based counterpart, making a clear, structured review especially valuable at this stage.

With this thesis, I aim to contribute by offering a comprehensive, practice-oriented review of the Python process mining ecosystem, with a special focus on PM4Py and its related extensions. All practical examples are provided as Jupyter notebooks, making it possible for others to reproduce and verify the results. This emphasis on reproducibility is important, as it ensures that the findings in Chapter 4 can be validated and built upon by future researchers and practitioners.

1.4 Methodology

This thesis combines a structured literature review with hands-on demonstrations. The work is organized into four main phases:

Phase 1: Building the theoretical foundation. In Chapter 2, I introduce and formalize the key mathematical concepts that underpin process mining. Most of these definitions are drawn from van der Aalst (Aalst, 2022). Starting with this theoretical groundwork helps establish a shared vocabulary for comparing different tools and for interpreting the practical examples that follow.

Phase 2: Surveying the tools. I identify relevant Python libraries through a mix of academic sources—such as publications from ICPM and related conferences—as well as searching the Python Package Index (PyPI), exploring GitHub repositories, and reviewing references within the PM4Py documentation. For each tool, I look at its main analytical focus, the data formats it can handle, the algorithms it implements,

and signs of active development and community support (such as recent updates and quality of documentation). The tools are grouped into five categories: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and supporting data processing packages.

Phase 3: Practical implementation. To show how these tools work in real scenarios, I developed five Jupyter notebooks that together demonstrate a complete process intelligence workflow. The first notebook introduces the basics using a small, hand-crafted dataset where the expected outcomes are clear. The remaining notebooks use the BPI Challenge 2019 procurement dataset and gradually build up the analysis: starting with process discovery and conformance checking, then adding variant analysis and throughput measurement, followed by performance analysis and object-centric directly-follows graph computation, and finally, process simulation and predictive modeling using a Random Forest classifier to flag cases likely to exceed a 30-day completion time. Each notebook combines executable Python code, outputs, and saved figures, ensuring that all results are fully reproducible.

Phase 4: Synthesizing findings and drawing conclusions. In Chapter 5, I bring together insights from the tool survey and the practical examples to answer the research question, discuss limitations, and suggest directions for future research.

It’s important to note that this thesis is descriptive and demonstrative rather than experimental or comparative. I do not conduct formal benchmarking or direct comparisons of algorithm performance, such as runtime or accuracy, as this would require a controlled experimental setup beyond the scope of this work. Instead, my contribution lies in organizing, explaining, and demonstrating the available tools in a way that is accessible and practical for both researchers and practitioners.

1.5 Structure of the thesis

The rest of this thesis is organized as follows:

Chapter 2 Lays out the mathematical foundations of process mining. Here, I define key concepts like events, traces, and event logs, and explain how directly-follows relations are used in process discovery. I also describe the main types of process models used in this thesis—directly-follows graphs, process trees, and Petri nets—and introduce the principles of conformance checking and performance analysis. This chapter also covers the Object-Centric Event Log (OCEL) format and takes an exploratory look at the BPI Challenge 2019 dataset.

Chapter 3 Reviews the primary Python tools available for process mining and process intelligence. The tools are grouped into five categories: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and complementary data processing packages. Each category is discussed in terms of the tasks it supports and the specific tools it includes. The chapter ends with a summary table that compares the main features of the reviewed tools.

Chapter 4 Provides a step-by-step walkthrough of five practical Jupyter notebooks, covering the full process intelligence workflow. The first notebook uses a synthetic event log to introduce core process mining ideas. The second and third notebooks apply these methods to real procurement data from the BPI Challenge 2019, resulting in a Petri net model, conformance scores, and throughput time distributions. The fourth notebook adds a performance perspective by annotating directly-follows graphs with median waiting times, measuring the level of automation, and computing object-type-specific DFGs from the OCEL relations. The fifth and final notebook looks ahead, using SimPy to simulate throughput times under different resource scenarios and applying a Random Forest model to predict whether a case will be completed quickly or slowly based on early features.

Chapter 5 Summarizes the main findings, discusses the thesis's limitations, and suggests directions for future research.

Chapter 2

The mathematics of process mining

To truly understand process mining, it's important that everyone speaks the same language. Without precise definitions, terms like “fitness” or “conformance” can mean different things to different people, making it difficult to compare tools or algorithms in a meaningful way. This chapter lays out the core mathematical concepts that will be used throughout the thesis: events, traces, event logs, directly-follows relations (and their variants), process discovery and modeling, conformance checking, and performance analysis. Each concept is illustrated with a simple example, so the formal definitions are always connected to something concrete. I'll also explain how event data are represented in Python and give a first look at the BPI Challenge 2019 dataset, which forms the basis for the case study in Chapter 4.

2.1 Event Logs and Traces

At the heart of process mining is the event — the most basic piece of data we work with. Each event captures the fact that a particular activity was carried out for a particular instance of a process, at a particular moment in time. More formally, let $\mathcal{U}_C$ be the set of all possible case identifiers, $\mathcal{U}_A$ the set of all activity names, and $\mathcal{U}_T$ the set of all timestamps, with $\mathcal{U}_T$ ordered chronologically.

An event e is a tuple $e = (c, a, t)$, where $c \in \mathcal{U}_C$ is the case identifier, $a \in \mathcal{U}_A$ is the activity name, and $t \in \mathcal{U}_T$ is the timestamp. In practice, events might also include extra attributes like the resource that performed the activity, the organizational unit, or other business-specific data. For most process discovery tasks, only these three are strictly necessary.

A trace is simply the sequence of activities that make up a single case, ordered by when they happened. If A is the set of all activity names, a trace σ is a sequence $\langle a_1, a_2, \dots, a_n \rangle$, with each $a_i \in A$, ordered by timestamp. The set of all possible traces is written as A^* .

An event log L collects the traces from many process cases. Since the same sequence of activities can happen in more than one case, the event log is modeled as a multiset over A^* , denoted $L \in \mathcal{B}(A^*)$, where $\mathcal{B}$ represents the multiset operator. A multiset is a function $m : X \rightarrow \mathbb{N}_0$ that tells us how many times each possible trace occurs in the log. For a trace $\sigma \in A^*$, the value $L(\sigma)$ gives the number of cases where that exact sequence was observed.

To make this more concrete, let's look at the following nine events:

Event ID	Activity	Case ID	Timestamp
e1	Create order	1	09:00
e2	Approve order	1	10:00
e3	Send invoice	1	11:00

Event ID	Activity	Case ID	Timestamp
e4	Create order	2	09:15
e5	Approve order	2	10:30
e6	Send invoice	2	11:20
e7	Create order	3	08:45
e8	Reject order	3	09:50
e9	Close case	3	10:40

Grouping by case identifier and ordering by timestamp gives three traces:

- Case 1: $\sigma_1 = \langle \text{Create order}, \text{Approve order}, \text{Send invoice} \rangle$
- Case 2: $\sigma_2 = \langle \text{Create order}, \text{Approve order}, \text{Send invoice} \rangle$
- Case 3: $\sigma_3 = \langle \text{Create order}, \text{Reject order}, \text{Close case} \rangle$

Since $\sigma_1 = \sigma_2$, the event log can be written as the multiset $L = [\sigma_1^2, \sigma_3^1]$ — the approval path occurs twice and the rejection path once. In total, the log contains three cases, two unique variants, and the average trace length is $\frac{3+3+3}{3} = 3$.

Before applying any process discovery algorithms, it is common practice to calculate a few summary statistics for the event log. These include:

- The number of distinct variants, which is simply the count of unique trace patterns that appear in the log.
- The average trace length, calculated as the total number of activities across all cases divided by the number of cases.
- The variant frequency distribution, which ranks each trace pattern by how often it occurs.

In real-world event logs, the frequency distribution of variants is usually very uneven: a small number of trace variants account for the majority of cases, while most variants occur only once or twice. This kind of skewed distribution is important to recognize, as it influences decisions about filtering and sampling before running discovery algorithms.

Analyzing traces is valuable for understanding both the structure and the variability of a process. If most cases follow the same sequence, the process is likely stable and predictable. But if there are many different variants, the process is more flexible, or potentially chaotic — and might need a more adaptable model to describe it effectively. That’s why, when analysts work with a new event log, one of the first things they look at is the distribution of trace frequencies: it provides a quick sense of how complex and variable the process really is.

2.2 Activities, variants, and directly-follows relations

One of the key ways to understand the structure of a process is by looking at the directly-follows relationship between activities in the event log. Simply put, activity b is said to directly follow activity a in a trace if a comes immediately before b in the sequence.

Formally, for a trace $\sigma = \langle a_1, \dots, a_n \rangle$, we say b directly follows a (written as $a \xrightarrow{\sigma} b$) if there is some position i in the trace such that $a_i = a$ and $a_{i+1} = b$.

To extend this idea to the entire event log L , we look at all traces and collect every pair of activities where one directly follows the other anywhere in any trace. This gives us the set of directly-follows pairs for the whole log:

$$\xrightarrow{L} = \bigcup_{\sigma \in \text{support}(L)} \xrightarrow{\sigma} \quad (1)$$

The **frequency** of a directly-follows pair (a, b) counts how many times b immediately follows a across all traces in the log:

$$f_{L(a,b)} = \sum_{\sigma \in L} L(\sigma) \cdot |\{i : a_i = a, a_{i+1} = b\}| \quad (2)$$

The directly-follows graph (DFG) is a useful way to visualize the flow of activities in an event log. For a given log L , the DFG is defined as a directed, weighted graph $\text{DFG}(L) = (A_L, E_L, f_L)$, where:

- A_L is the set of activities that appear in at least one trace,
- E_L is the set of directed edges representing directly-follows relationships between activities, and
- f_L assigns a frequency to each edge, indicating how often a particular directly-follows pair occurs in the log.

To give a complete picture, the DFG also includes a special start node and end node. The start node connects to every activity that begins at least one trace, and every activity that ends a trace connects to the end node. The weights on these edges reflect how often each activity starts or ends a trace.

Using the event log from the previous example, we can list all the directly-follows pairs and count how many times each one appears:

From activity	To activity	Frequency
Create order	Approve order	2
Approve order	Send invoice	2
Create order	Reject order	1
Reject order	Close case	1

The DFG reveals two distinct execution paths: the approval path (Create $\rightarrow$ Approve $\rightarrow$ Send, frequency 2) and the rejection path (Create $\rightarrow$ Reject $\rightarrow$ Close, frequency 1). This already suggests an exclusive choice after Create order — something a discovery algorithm would later formalize into a proper model. The DFG gives a quick visual overview of which activities tend to follow each other and how often, which makes it a useful first step in any process analysis. That said, it does not distinguish between control-flow patterns such as sequences, choices, and parallel branches, which limits how far you can go with it on its own.

A variant is any trace σ with $L(\sigma) > 0$. The variant set is $\text{Var}(L) = \{\sigma \in A^* : L(\sigma) > 0\}$. Variant analysis ranks variants by frequency and examines the proportion of cases accounted for by the top- k variants. The coverage of the top k variants is:

$$\text{Coverage}(k) = \frac{\sum_{i=1}^k L(\sigma_i^*)}{\|L\|} \quad (3)$$

where $\sigma_1^*, \sigma_2^*, \dots$ are the variants ordered by decreasing frequency.

Beyond variants, simple summary counts are also worth looking at early on — the number of events per activity, the total number of cases, the number of distinct variants, and the average trace length. These give the analyst a quick sense of the overall complexity and variability of the process before moving on to more detailed techniques.

2.3 Process Discovery and Process Models

Process discovery is the task of building a process model directly from an event log, without relying on any pre-existing reference model. A good discovered model needs to balance four quality dimensions:

- **Fitness** measures whether the model can reproduce the behavior observed in the log. A model with high fitness accepts most of the traces that actually occurred — ideally all of them. Formally, if $\mathcal{L}(M)$ denotes the language (the set of accepted traces) of model M , then high fitness means $\text{Var}(L) \subseteq \mathcal{L}(M)$, or at least most of $\text{Var}(L)$ is covered.
- **Precision** measures whether the model avoids accepting behavior that was never observed. A model with low precision is too permissive — it allows many traces that never appeared in the log. A model that simply accepts any sequence of any activity would have perfect fitness but zero precision.
- **Generalization** reflects how well the model handles cases that were not seen during discovery but are still plausible given the process structure. A model that memorizes every observed trace exactly — without allowing for any unseen but valid behavior — is said to overfit the log.
- **Simplicity** refers to how easy the model is to read and interpret. Simpler models are generally preferred when they still achieve acceptable fitness and precision. The most common simplicity measure is the total number of nodes and arcs in the model.

These four dimensions are in tension with each other. Improving fitness often comes at the cost of precision, since a more permissive model accepts a wider range of behavior. Finding the right balance is one of the central challenges in process discovery, and different algorithms handle this trade-off differently. For large and noisy real-world logs, robustness to noise and computational efficiency are also practical factors that influence which algorithm to use.

2.3.1 The Alpha algorithm

The Alpha algorithm is one of the earliest and most well-known process discovery algorithms. It works by analyzing the directly-follows relations in the log and inferring causal dependencies between activities, then using those dependencies to construct a Petri net.

The algorithm starts by computing four binary relations between all pairs of activities in A_L :

- $a \rightarrow b$ (direct succession): b directly follows a in at least one trace
- $a \gg b$ (causality): $a \rightarrow b$ holds but $b \rightarrow a$ does not — a appears to cause b
- $a \parallel b$ (parallelism): both $a \rightarrow b$ and $b \rightarrow a$ hold — the two activities can appear in either order
- $a \# b$ (independence): neither $a \rightarrow b$ nor $b \rightarrow a$ holds — the activities never directly follow each other

From these relations, the algorithm identifies all pairs (A, B) of non-empty activity sets where every activity in A causes every activity in B (i.e., $a \gg b$ for all $a \in A, b \in B$), all activities within A are independent of each other, and all activities within B are independent of each other. The maximal such pairs become places in the Petri net, each connecting the activities in A as input transitions to the activities in B as output transitions.

To illustrate, consider the log $L = [\langle A, B, D \rangle^4, \langle A, C, D \rangle^3]$.

The directly-follows pairs are $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$, which gives the causal relations $A \gg B$, $A \gg C$, $B \gg D$, $C \gg D$. Since neither $B \rightarrow C$ nor $C \rightarrow B$ holds, we have $B \# C$.

The maximal causal pairs are $(\{A\}, \{B, C\})$, $(\{B\}, \{D\})$, and $(\{C\}, \{D\})$. Adding an initial place connected to A and a final place connected from D , the result is a Petri net where A leads to an exclusive choice between B and C (since $B \# C$), and both paths converge at D . This correctly represents both variants in the log.

The Alpha algorithm is historically important — it was the first to show that a process model can be derived automatically from event data alone. That said, it has well-documented limitations: it handles loops poorly, is sensitive to noise and infrequent behavior, cannot represent all control-flow patterns correctly, and fails on logs where certain activity combinations violate its structural assumptions. More robust algorithms have since been developed to address these weaknesses.

2.3.2 The Inductive Miner and Process Trees

The Inductive Miner is a more recent and widely used discovery algorithm that addresses many of the limitations of the Alpha algorithm. It works by recursively dividing the event log into smaller sublogs based on detected control-flow patterns — sequences, exclusive choices, parallel branches, and loops. Each subdivision corresponds to a node in a **process tree**.

A process tree is a hierarchical model in which leaf nodes represent individual activities (or the silent activity τ , which represents an internal step with no observable name) and internal nodes are **operator nodes** that define how their children are executed. The four main operators are: sequence ($\rightarrow$), exclusive choice ($\times$), parallel execution ($+$), and loop ($\circ$). More precisely:

- $\rightarrow (T_1, T_2, \dots, T_k)$: execute the sub-trees $T_1, T_2, \dots, T_k$ in strict left-to-right order.
- $\times (T_1, T_2, \dots, T_k)$: execute exactly one of the sub-trees, chosen exclusively.
- $+(T_1, T_2, \dots, T_k)$: execute all sub-trees, but in any interleaved order (true parallelism).
- $\circ(T_{\text{body}}, T_{\text{redo}})$: execute the body T_{body} , then optionally execute the redo branch T_{redo} and repeat the body, until the loop is exited.

Process trees always produce **sound** models, meaning they cannot result in deadlocks or incomplete executions. Every process tree can be automatically converted into a corresponding Petri net, so the two representations are interchangeable for analytical purposes.

The Inductive Miner algorithm discovers a process tree using a divide-and-conquer strategy. Starting with the full log, it attempts to detect a **cut** — a partition of activities into groups that correspond to one of the four operators. If a sequence cut is detected, the log is split into sublogs for each group and the algorithm recurses. If an exclusive choice cut is detected, the log is split by separating cases that contain each group’s activities. If a parallel cut is detected, the log is projected onto each group. If a loop cut is detected, the body and redo sublogs are separated. If no cut is detected, a fall-through strategy is applied.

The **Inductive Miner Infrequent (IMf)** variant, which is used in this thesis via PM4Py, filters out infrequent directly-follows pairs before attempting to detect cuts. This makes the algorithm more robust to noise and exceptional cases in large real-world logs, at the cost of some fitness on rare variants.

To illustrate, consider $L = [\langle A, B, D \rangle^4, \langle A, C, D \rangle^3]$. The algorithm first detects a sequence cut: A is always first and D is always last, so the partition $A_1 = \{A\}$, $A_2 = \{B, C\}$, $A_3 = \{D\}$ is valid. Recursing on $A_2 = \{B, C\}$: the sub-log is $[\langle B \rangle^4, \langle C \rangle^3]$ and no directly-follows relation exists between B and C , so an exclusive choice cut applies. The final process tree is $\rightarrow (A, \times (B, C), D)$: always start with A , exclusively choose B or C , then always end with D . This correctly represents both variants in the log and produces a sound Petri net automatically.

2.3.3 Petri Nets

The main model representation used in this thesis is the Petri net, which is also the standard output format for PM4Py and most other process mining tools. A Petri net provides a flexible and formal way to describe how activities in a process are connected and how cases move through the process.

Formally, a Petri net is defined as a triple $N = (P, T, F)$, where:

- P is a finite set of **places** (usually drawn as circles),

- T is a finite set of **transitions** (usually shown as rectangles), with P and T being disjoint ($P \cap T = \emptyset$),
- $F \subseteq (P \times T) \cup (T \times P)$ is the set of directed arcs, connecting places to transitions and transitions to places.

For any transition $t \in T$:

- The **preset** $\bullet t$ is the set of input places (all places with an arc going to t),
- The **postset** $t \bullet$ is the set of output places (all places with an arc coming from t).

A **marking** M assigns a non-negative integer (a number of tokens) to each place. The combination (N, M) is called a marked Petri net. A transition t is **enabled** in marking M if every input place of t has at least one token. When t fires, it consumes a token from each input place and produces a token in each output place, resulting in a new marking.

An **accepting Petri net** specifies an initial marking M_0 and a final marking M_f . A trace $\sigma = \langle a_1, \dots, a_n \rangle$ is accepted if there is a sequence of transition firings, matching the labels in σ , that leads from M_0 to M_f .

As a simple example, consider a sequential process $A \rightarrow B \rightarrow D$:

- Places: $P = \{p_0, p_1, p_2, p_3\}$,
- Transitions: $T = \{A, B, D\}$,
- Flow: $F = \{(p_0, A), (A, p_1), (p_1, B), (B, p_2), (p_2, D), (D, p_3)\}$,
- Initial marking: $M_0 = \{p_0 : 1\}$, Final marking: $M_f = \{p_3 : 1\}$.

If we replay the trace $\langle A, B, D \rangle$:

- Step 1: A is enabled because p_0 has a token. Firing A moves the token to p_1 .
- Step 2: B is enabled because p_1 now has a token. Firing B moves the token to p_2 .
- Step 3: D is enabled because p_2 has a token. Firing D moves the token to p_3 , which matches the final marking. This trace is accepted.

If we try to replay $\langle A, D \rangle$: after firing A , the token is at p_1 . D requires a token at p_2 , but p_2 is empty, so D is not enabled and the trace cannot be completed. This correctly shows that B cannot be skipped in this model.

In process mining, transitions represent activities, places represent the state between activities, and tokens represent the current execution state of a process instance. Petri nets are highly valuable because they are both easy to visualize and allow for formal analysis of properties like reachability and soundness. They are also the primary model format used by PM4Py for tasks such as conformance checking and performance analysis.

2.4 Conformance Checking

Once you have both a process model and an event log, you can compare them to see how closely real-life process executions match the intended process design. This comparison, known as **conformance checking**, is one of the most practically valuable applications

of process mining. It allows organizations to spot rule violations, unexpected deviations, and potential data quality issues.

There are two main approaches to conformance checking:

1. Token-based replay

This method simulates each trace from the log on the Petri net model by moving tokens according to the observed activity sequence. If a trace follows a path allowed by the model, the replay goes smoothly. If not, extra tokens might need to be added (missing tokens), or some tokens may be left behind at the end (remaining tokens). The fitness of a single trace is calculated as:

$$\text{fitness}(\sigma) = \frac{1}{2} \left(1 - \frac{p}{c} \right) + \frac{1}{2} \left(1 - \frac{r}{q} \right) \quad (4)$$

where:

- p = number of missing tokens added,
- c = total number of consumed tokens,
- r = number of remaining tokens at the end,
- q = total number of produced tokens.

A fitness value of 1 means the trace fits the model perfectly; lower values indicate greater deviation. The overall fitness for the log is measured as the weighted average of all trace fitness values (Aalst, 2022).

Example: Consider the process model $A \rightarrow B \rightarrow D$ and the trace $\langle A, D \rangle$ (activity B is skipped). After firing A , there is a token at p_1 . D requires a token at p_2 , which is not present, so one missing token is added ($p = 1$). After firing D , there is still a token left at p_1 ($r = 1$). With $c = 2$ and $q = 2$:

$$\text{fitness} = \frac{1}{2} \left(1 - \frac{1}{2} \right) + \frac{1}{2} \left(1 - \frac{1}{2} \right) = 0.5 \quad (5)$$

This reflects a significant deviation: one activity was skipped, and one token was left unconsumed.

2. Alignment-based conformance

This approach finds the closest valid execution in the model for each observed trace. An **alignment** is a sequence of move pairs (a_i, b_i) , where a_i is a move in the log and b_i is a move in the model:

- **Synchronous move** (a, a) : Activity occurs in both log and model — no deviation.
- **Log move** $(a, \gg)$: Activity occurs in the log but not in the model at this point — unexpected behavior.
- **Model move** $(\gg, a)$: Activity expected by the model but missing from the log — expected behavior was skipped.

The cost of an alignment is the number of non-synchronous moves, and the optimal alignment minimizes this cost, usually found by solving a shortest-path problem (commonly with the A^* algorithm).

Example: Comparing $\langle A, D \rangle$ to the model accepting $\langle A, B, D \rangle$:

Log move	Model move	Type
A	A	Synchronous
>>	B	Model move — B expected but not observed
D	D	Synchronous

The alignment cost is 1, pinpointing that the only deviation is the absence of activity B . This is more informative than a fitness score alone, as it shows exactly what was missed and where.

Alignment-based conformance gives much more detailed diagnostic information than token replay, but it is also more computationally intensive, especially for long traces and complex models. In practice, token replay is often used for a quick, overall fitness check, while alignment-based conformance is preferred when detailed, trace-level diagnostics are needed.

Deviations found through conformance checking can point to data quality problems, rare exceptions, policy violations, or even real changes in how the process is carried out over time. Understanding what kinds of deviations occur — and how often — is a central outcome of any conformance checking study.

2.5 Performance Analysis and Process Intelligence

Performance analysis adds a crucial time dimension to the structural insights obtained from process discovery and conformance checking. Because event logs include time-stamps, it is possible to measure how long each activity takes, how much time passes between activities, and the overall time it takes to complete a case. By projecting these measurements onto the process model, we can create a performance-annotated view that highlights exactly where time is spent — or lost — in the process.

The **throughput time** of a case c is simply the time between its first and last recorded event:

$$TT(c) = \max_{e \in c} t(e) - \min_{e \in c} t(e) \quad (6)$$

We can also calculate the **waiting time** between two consecutive activities a_i and a_{i+1} in a trace, and the **service time** for an activity (if both its start and end are recorded).

Common summary statistics for throughput time include:

- **Mean throughput time:** Average duration for all cases.
- **Median throughput time:** Middle value, less sensitive to outliers.
- **P95 throughput time:** 95% of cases finish within this time.
- **Minimum and maximum:** Fastest and slowest observed cases.

- **Coefficient of variation:** Standard deviation divided by the mean; higher values indicate more variability.

The coefficient of variation is particularly useful — a low value means cases are fairly consistent, while a high value signals significant variation, which might point to bottlenecks or special cases.

When these time measurements are mapped onto the process model — tagging each step with statistics like median or mean duration — we get a performance-annotated model. This visualization makes it easy to spot where delays occur. For instance, if two activities are always directly connected in the model but there is a consistent delay between them in the data, this suggests a possible bottleneck.

Process intelligence takes things further by combining process data with forecasting, simulation, and optimization. Once we have established a performance baseline, we can start to predict outcomes for ongoing cases, simulate the effects of potential process changes, or build dashboards that alert managers when a case is likely to exceed a target time. The mathematical concepts introduced in this chapter remain relevant for these more advanced, forward-looking applications — not just for retrospective analysis.

2.6 Event Data Representation in Python

From an implementation perspective, event data in Python are usually stored as either standard tables (DataFrames) or as more specialized log objects. The simplest approach is a table with three columns: case identifier, activity label, and timestamp. This format works well with the `pandas` library, which is the standard data structure for Python data science.

Process mining libraries like PM4Py can convert these tables into richer event-log objects that preserve the structure of traces and support various discovery algorithms. In PM4Py, you first use `pm4py.format_dataframe` to specify which columns correspond to the case ID, activity, and timestamp, then `pm4py.convert_to_event_log` to create the internal log object. This object then supports all of PM4Py’s discovery, conformance, and performance functions. This conversion is demonstrated in the synthetic example at the start of Chapter 4, where a simple DataFrame is transformed for use with the Inductive Miner.

A more advanced format is the **Object-Centric Event Log (OCEL)**. Unlike traditional logs, where each event is linked to just one case, OCEL allows each event to be associated with multiple objects of different types. For example, a procurement event might relate to a purchase order, an order item, a vendor, and a resource all at once. The OCEL standard defines how to store and handle such logs, and PM4Py provides `pm4py.read_ocel` to read OCEL files in JSON format. This produces an object with separate DataFrames for events and objects, plus metadata about object types and event attributes.

Flattening is the process of turning an OCEL into a classical event log by selecting one object type to serve as the case identifier. In PM4Py, this is done with

`pm4py.ocel_flattening(ocel, object_type)`. If an event is linked to multiple objects of that type, it will appear in multiple cases in the flattened log, increasing the case count. If some events are not linked to the selected object type, they may be lost during flattening. Managing these effects is an important preprocessing step before using classical discovery algorithms on object-centric data.

In the practical examples in Chapter 4, both representations are used. The introductory example uses the simple tabular format to show the basic conversion process. The BPI Challenge 2019 case study uses the OCEL format, and the flattening step demonstrates the preprocessing needed for classical analysis. Later examples return to the OCEL format directly, computing an object-centric directly-follows graph from the relations table without flattening, so that differences in process behavior by object type can be compared without the distortions introduced by flattening.

2.7 Exploratory Description of the BPI Challenge 2019 Dataset

For the real-world case study in this thesis, I use the BPI Challenge 2019 dataset, which is stored in OCEL format. This dataset was collected from the procurement process of a large multinational company and was released as part of the annual BPI Challenge, organized alongside the International Conference on Process Mining. It has become a widely used benchmark in academic research and is a representative example of what large, complex, real-world event logs look like.

The dataset includes roughly 1.6 million events and over 330,000 objects, which are divided into four types: purchase orders (PO), purchase order items (POItem), resources, and vendors. Each event comes with a timestamp, an activity name, and a rich set of business context attributes — such as company code, document type, item category, spending area, and vendor information. This level of detail makes the dataset valuable not only for analyzing process flows, but also for performance analysis and organizational mining.

Characteristic	Value
Total events	1,595,923
Total objects	330,685
Object types	4 (PO, POItem, Resource, Vendor)
Format	OCEL (JSON)
Domain	Procurement

To analyze the process from a case-centric perspective, I flattened the log using the POItem object type, since this is the most common type in the data. After flattening, each case corresponds to a unique POItem. In the full dataset, this results in a very large number of cases; for the practical examples in Chapter 4, I use a sample of 53,198 cases drawn from the flattened log.

Metric	Value
Cases with at least 2 events	248,899
Mean throughput time	72.3 days
Median throughput time	64.3 days
95th percentile	143.0 days
Minimum	0.0 days
Maximum	25,670.6 days

On average, each purchase order item takes about two months from the first to the last recorded event. The 95th percentile — 143 days — shows that while most cases are handled in a predictable time frame, a minority take much longer. The extreme maximum (over 25,000 days) is almost certainly an artifact, perhaps representing an open case that was never formally closed or a data entry error. The distribution of throughput times is skewed to the right, which is typical for procurement processes where most cases finish on schedule but some outliers drag on far longer.

These statistics highlight several important features of the dataset. First, its size means that efficient tools and careful preprocessing are essential before analysis. Second, having four object types means the process cannot be fully captured with a simple, single-case model — some relational information is inevitably lost if we try. Third, the combination of detailed process data and rich business context makes this dataset suitable for a wide range of process mining tasks, not just process discovery.

In Chapter 4, I show how to load this dataset using `pm4py.read_ocel`, flatten it to a case-based format, and sample it down to a manageable size before running any discovery algorithms. The theoretical concepts discussed in this chapter — traces, variants, Petri nets, and throughput time — are all put into practice there, connecting the mathematical foundations with real analytical results.

Chapter 3

The Tools Available in Python

Over the past decade, the Python ecosystem for process mining and process intelligence has grown rapidly. Today, Python supports a wide range of analytical tasks — from loading and exploring event logs, to discovering process models, checking conformance, predicting future outcomes, and even simulating alternative process designs. This chapter provides a structured overview of the main Python tools available for these purposes. For clarity, the tools are grouped into five categories: core process mining libraries, predictive and AI-oriented tools, simulation and optimization tools, visualization libraries, and supporting data processing packages. For each tool, I discuss its primary purpose, the data formats it supports, the main algorithms or methods it implements, and its practical strengths and limitations. At the end of the chapter, there is a comparative summary table for quick reference.

The selection of tools covered here is based on a review of academic publications (especially from the International Conference on Process Mining), the Python Package Index (PyPI), and active GitHub repositories. The focus is on tools that are actively maintained, widely used in the community, or that represent an important category of functionality. Commercial tools are only included if they provide a Python interface that is accessible to practitioners, such as PyCelonis.

3.1 Core Process Mining Libraries

The backbone of Python process mining is formed by a handful of core libraries. These libraries implement the essential workflow: reading in event data, exploring log statistics, discovering process models, checking conformance, and analyzing process performance. In this section, I describe the four main libraries in this category: PM4Py, PM4Py-Streaming, Simod, and PMLab.

3.1.1 PM4Py

PM4Py is the central library in the Python process mining ecosystem. Developed and maintained by the Process Intelligence Solutions group (with academic roots at Fraunhofer FIT and RWTH Aachen University), PM4Py was first released in 2018. Since then, it has become the de facto standard for process mining in Python, with regular updates, active development, and a growing user base in both academia and industry.

PM4Py covers the entire process mining workflow. Its main capabilities include:

- **Event log input and output:** PM4Py can read and write event logs in various formats, including XES (the IEEE standard), CSV files via pandas DataFrames, and OCEL JSON files for object-centric logs. Reading a standard XES file is as simple as calling `pm4py.read_xes()`, while `pm4py.read_ocel()` is used for OCEL logs. Both

functions return structured objects that integrate directly with the rest of PM4Py’s analysis functions.

- **Log statistics and exploration:** PM4Py offers functions for computing basic statistics, such as the number of activities, cases, and events, as well as variant frequencies, start and end activities, and directly-follows relations. For example, `pm4py.get_variants()` retrieves variant counts, and `pm4py.discover_dfg()` constructs the directly-follows graph. These functions work with both classical event logs and OCEL structures.
- **Process discovery:** Several discovery algorithms are implemented. The Alpha algorithm, available via `pm4py.discover_petri_net_alpha()`, creates a Petri net from directly-follows relations. The Inductive Miner (`pm4py.discover_petri_net_inductive()`) produces a sound Petri net through a process tree, and is the main algorithm used in my case study in Chapter 4. The Heuristics Miner (`pm4py.discover_petri_net_heuristics()`) uses frequency thresholds to filter the directly-follows graph before building the model, making it more robust to noise than the Alpha algorithm. All discovery functions accept either a classic event-log object or a pandas DataFrame with specified column names.
- **Conformance checking:** PM4Py provides both token-based replay (`pm4py.fitness_token_based_replay()`) — which is fast and suitable for large logs — and alignment-based conformance (`pm4py.fitness_alignments()`), which gives detailed per-trace diagnostics at a higher computational cost. Both return a dictionary of fitness metrics, including the overall fitness score.
- **Performance analysis:** PM4Py calculates throughput times, activity durations, and waiting times for event logs with timestamps. These statistics can be projected onto a process model to generate performance-annotated visualizations that highlight bottlenecks.
- **Object-centric process mining:** PM4Py is currently the only Python library with comprehensive, native support for OCEL. It provides `pm4py.ocel_flattening()` to convert an OCEL log to a classical format, and `pm4py.discover_ocdfg()` to compute object-centric directly-follows graphs without flattening.

Strengths and limitations: PM4Py’s main strengths are its wide coverage, active development, good documentation, and robust OCEL support. It is the only Python library that offers a complete process mining workflow in a single package. The main limitations are computational: alignment-based conformance and some discovery algorithms can be slow on very large logs, and certain advanced features — like simulation — are beyond its current scope.

3.1.2 PM4Py-Streaming

PM4Py-Streaming is an extension of the core PM4Py library, designed specifically for situations where event data arrives continuously — as a stream — rather than in a complete, stored log. This is particularly useful in real-world scenarios like live

monitoring of ERP systems or logistics platforms, where new events are generated in real time. In these cases, analysts need to keep the process model up to date without having to reprocess all historical events each time something new happens.

The library implements online versions of key process mining algorithms. These algorithms update their internal state incrementally with each incoming event, without storing the full event history. For example, the online directly-follows graph keeps running frequency counts for each activity pair and updates them instantly as new events come in. There is even an online variant of the Inductive Miner, which can adapt the discovered process tree as stream statistics change over time. PM4Py-Streaming is designed to work seamlessly with PM4Py’s standard data structures, so any models discovered from the stream can be further analyzed using the conformance and performance functions in PM4Py.

Strengths and limitations: PM4Py-Streaming is currently the only Python library dedicated to online process mining. Its main limitation is that it supports fewer algorithms than the standard batch PM4Py library, and the approximations required for online discovery can sometimes reduce model quality compared to what is possible with a complete, static log.

3.1.3 Simod

Simod is a Python tool focused on simulation-based optimization of business processes. It takes a BPMN process model — usually one produced by process discovery — as input, estimates simulation parameters from an event log, and then generates an optimized simulation configuration. The ultimate goal is to identify the combination of resource assignments, activity durations, and scheduling rules that will minimize some target objective, such as overall throughput time or cost.

Simod’s workflow proceeds in three main phases. First, it extracts and converts a process model from the event log to BPMN format. Next, it estimates simulation parameters — such as resource pools, activity duration distributions, and arrival rates — from the log data. Finally, a Bayesian optimization loop runs multiple simulation experiments with different parameter settings, searching for the configuration that best achieves the chosen objective.

Strengths and limitations: Simod fills an important gap in the Python ecosystem by linking process mining with simulation-based optimization. However, it requires a fairly clean and well-structured BPMN model as input, so it is most useful after a successful process discovery step has already been completed. It also depends on tools like Graphviz and a BPMN simulation engine, which can add to the installation and setup complexity.

3.1.4 PMLab

PMLab is a research-oriented toolkit for process mining in Python. It provides implementations of a variety of process discovery and conformance algorithms, including several that are not available in PM4Py. PMLab is aimed mainly at researchers who

want to experiment with or develop new process mining algorithms, rather than practitioners who need a polished, production-ready tool.

The library supports classical event logs in XES format and offers Python implementations of the Alpha algorithm, several variants of the Heuristics Miner, and various conformance checking methods. PMLab is designed to be modular and extensible, making it easier for researchers to add or modify algorithms without having to overhaul the entire library.

Assessment: PMLab is a valuable resource for researchers who need to dive deep into algorithm development and modification. However, it is less suited for production use — development has slowed, documentation is limited, and it lacks support for newer features like OCEL and the Inductive Miner Infrequent variant.

3.1.5 BupaR (Python Interface)

BupaR is a comprehensive process mining framework originally developed for the R programming language (Janssenswillen *et al.*, 2019). It introduces a structured grammar for process analysis — a set of consistent operations for filtering, aggregating, and visualizing event data — centered around a dedicated event log object. This approach ensures that all analyses are built on a consistent data structure. BupaR’s expressive and readable pipeline syntax has influenced several tools in the Python ecosystem and remains popular in academic research for its clarity and reproducibility.

Python users can access BupaR’s functionality through the rpy2 bridge, which allows R functions to be called directly from within a Python environment. This means analysts can leverage BupaR’s powerful event log manipulation and visualization tools — such as dotted charts and process maps — without leaving Python. These visualizations are widely seen in academic papers as clear, succinct summaries of process behavior.

Strengths and limitations: BupaR’s greatest strength is its expressive, consistent grammar for process analysis, which supports a broader range of log manipulation and visualization operations than PM4Py’s current Python API. Its main limitation for Python users is the need for a working R installation and the rpy2 bridge, which adds a layer of complexity compared to pure-Python solutions.

3.2 Predictive and AI-Oriented Tools

A rapidly growing set of Python tools connects process mining with machine learning and artificial intelligence, enabling predictive process monitoring, automated decision support, and intelligent process automation. The following are some of the most prominent libraries in this area.

3.2.1 ML4ProM

ML4ProM is a Python framework that brings together process mining techniques and machine learning models to support predictive process monitoring. This area focuses on forecasting future states for running process instances — for example, predicting

whether a case will finish on time, what the next activity will be, or how long a case is likely to take.

The ML4ProM workflow follows a standard pattern: first, the event log is preprocessed to extract features from partial traces. These features are then used to train a machine learning model, such as a classifier or regressor from scikit-learn. The trained model can then be applied to new, ongoing cases to generate predictions. ML4ProM provides several feature extraction strategies that are commonly used in process mining, including sequence encoding (representing the activity prefix as a fixed-length vector), last-state encoding (capturing the most recent activity and its attributes), and aggregate encoding (using statistics over the trace prefix).

Choosing the right feature encoding is important, as it determines how much of the process history is captured and how well the features represent the process state. ML4ProM enables analysts to experiment with different encodings and compare their predictive accuracy using standard metrics like accuracy, AUC-ROC, and mean absolute error.

Strengths and limitations: ML4ProM’s main strength is its seamless integration of process mining feature extraction with scikit-learn-compatible models. Its main drawbacks are the need for a reasonably large labeled training dataset and the current lack of support for object-centric event logs.

3.2.2 PM4PYML

PM4PYML is an extension of PM4Py that directly integrates machine learning pipelines with PM4Py event-log objects and analysis functions. It provides scikit-learn-compatible transformers to convert PM4Py event logs into feature matrices, making it easy to train machine learning models for predictive process monitoring within the PM4Py ecosystem.

The library supports several feature extraction methods that work with PM4Py’s internal log format, including counts of activity occurrences, frequencies of directly-follows pairs, and time-based features like remaining and elapsed time. These features can be plugged into any scikit-learn estimator using standard pipeline objects. The `LogTransformer` class converts a PM4Py event log directly into a NumPy feature matrix, which can then be passed to any scikit-learn classifier or regressor.

Strengths and limitations: PM4PYML’s main advantage is its tight integration with PM4Py, which streamlines the process of going from process mining analysis to machine learning modeling. Its main limitations are similar to ML4ProM: it requires labeled training data and does not natively support object-centric logs.

3.2.3 PyCelonis

PyCelonis is the official Python API for the Celonis Process Intelligence platform — a leading commercial solution used by large organizations for process monitoring, compliance, and improvement at scale. With PyCelonis, analysts can interact with the

Celonis platform directly from Python, making it possible to automate analyses, extract data and results, and integrate Celonis insights into broader data science workflows.

Key features of PyCelonis include connecting to a Celonis instance (using an API key for authentication), extracting event log data using PQL (Process Query Language), sending analysis results and action recommendations back to the platform, and integrating Celonis with robotic process automation (RPA) tools for automated process execution.

Strengths and limitations: PyCelonis’s biggest strength is its ability to bridge Celonis’s powerful commercial process intelligence infrastructure with the flexibility of Python’s data science ecosystem. The main limitation is that it is tightly tied to the Celonis platform, which requires a commercial subscription — so it is not accessible for academic or small-scale projects without an institutional license.

3.2.4 BPMN-Python and Workflow Automation

The BPMN-python library and related tools enable users to extract, manipulate, and visualize BPMN process models in Python. BPMN (Business Process Model and Notation) is the industry standard graphical language for representing business processes, widely adopted in workflow automation systems. PM4Py can export discovered Petri nets to BPMN format, after which bpmn-python can import these models, modify them programmatically, or render them as interactive visualizations. This creates a practical bridge between process mining analysis and operational business process management systems.

Strengths and limitations: BPMN-python and similar tools are most valuable in organizations that already use BPMN-based workflow systems. For purely analytical purposes, however, they may add unnecessary complexity compared to the Petri net and process tree models already supported by PM4Py.

3.2.5 Predictive Process Monitoring Toolkit (PPMT)

The Predictive Process Monitoring Toolkit (PPMT) is a Python implementation of the benchmark framework for predictive process monitoring research described by Teinemaa et al. (Teinemaa *et al.*, 2019). It supports the full experimental pipeline: reading XES event logs, extracting prefixes of various lengths, applying multiple feature encoding strategies (such as index, sequence, and aggregate encoding), training a range of machine learning models (including logistic regression, random forest, and XGBoost), and evaluating predictive performance using metrics like AUC-ROC and mean absolute error.

PPMT is primarily a research tool, designed to reproduce and extend the benchmark results of Teinemaa et al. Its main value is that it allows researchers to compare new predictive methods against a well-established baseline, using a consistent experimental protocol — without having to build the evaluation infrastructure from scratch.

Strengths and limitations: PPMT’s main strength is its comprehensive benchmark coverage: it enables researchers to reproduce published results and add new methods

for standardized comparison. Its limitations are that it is not designed for real-time deployment, documentation is limited beyond the associated research paper, and it is not actively maintained as a production system.

3.3 Simulation and Optimization Tools

Simulation and optimization tools extend the reach of process mining from simply understanding what happened in the past to designing and testing improvements for future processes. The next section introduces the main Python tools for simulation and optimization: SimPy, BPSim, and ProcessOptimizer.

3.3.1 SimPy

SimPy is a popular Python library for discrete-event simulation. In this approach, a system is modeled as a series of events that happen at specific points in time, each of which can change the system’s state and schedule future events. This method fits business processes very well, since activities are performed by resources over time and cases enter and leave the process according to statistical patterns.

While SimPy is not designed specifically for process mining, it works well in combination with process mining results to simulate how a business process might behave under different conditions. The typical workflow starts with process mining: the event log is analyzed to discover the process structure and estimate activity durations. These parameters are then used to build a SimPy simulation, defining process generators, resource pools, and arrival rates. Finally, the simulation is run to generate synthetic event data for different process scenarios, which can be further analyzed using PM4Py.

For example, modeling a sequential process with three activities in SimPy involves defining a function that requests a resource, waits for a service time (often modeled as an exponential distribution), and releases the resource after each activity. By running this simulation with different numbers of available resources, it is possible to estimate how staffing changes might affect throughput time — before making any changes to the actual process.

Assessment: SimPy is simple, well-documented, and widely used, which makes it a practical tool for this kind of analysis. The main limitation is integration: there is no automatic bridge between a process model discovered with PM4Py and a SimPy simulation. For simple processes, building this bridge manually is straightforward, but it can become time-consuming as model complexity increases.

3.3.2 BPSim

BPSim is a simulation standard designed to work with BPMN process models. It defines a structured, XML-based format for specifying simulation parameters — such as resource pools, activity duration distributions, and routing probabilities — directly inside a BPMN model file. Tools that support BPSim, including Simod, can read these parameters and run structured simulation experiments without requiring custom code.

BPSim is especially useful when a process model is already available in BPMN format and the goal is to systematically evaluate multiple alternative configurations.

3.3.3 Process Optimizer

Process Optimizer is a Python library for Bayesian optimization, developed by Novo Nordisk Research. Built on top of scikit-optimize, it provides a framework for tuning process parameters in situations where each evaluation is expensive — such as when running a detailed simulation.

In the context of process mining, Process Optimizer can be used to find the combination of simulation parameters that minimizes throughput time or maximizes resource utilization. The analyst defines an objective function: it takes a set of process parameters, runs the simulation, and returns a performance score. Process Optimizer then suggests parameter settings, evaluates them, and builds a probabilistic model of the objective function to focus subsequent evaluations on the most promising regions of the parameter space.

Strengths and limitations: The main advantage of Process Optimizer is its efficiency: it can find good parameter settings without needing to try every possible combination, which is especially valuable when each simulation run is time-consuming. Its limitation is that the analyst must define a suitable objective function and simulation model ahead of time, so it is most useful after the process structure has already been established through discovery.

3.4 Visualization Libraries

Visualization plays a crucial role in process analysis, both for exploring event data and for communicating findings to stakeholders. The Python process mining ecosystem offers several libraries to support different types of process visualizations.

3.4.1 Graphviz

Graphviz is an open-source tool for graph visualization and serves as the primary rendering backend for process models in PM4Py. When a model — such as a Petri net, process tree, or directly-follows graph — is discovered, PM4Py automatically converts it to Graphviz’s DOT format and calls the `dot` command-line tool to generate images. This means analysts do not have to write any DOT code themselves. The resulting images can be saved as PNG or PDF files for reports, or displayed interactively in Jupyter notebooks.

In this thesis, Graphviz is used to visualize the Petri net models discovered from the BPI Challenge 2019 dataset. If Graphviz is not installed, PM4Py falls back on a built-in renderer for directly-follows graphs, but full Petri net visualization requires Graphviz to be available in the system PATH.

Strengths and limitations: Graphviz produces high-quality, publication-ready visualizations. Its main drawback is that it must be installed separately from PM4Py as an

external dependency, which can occasionally cause installation or compatibility issues on some systems.

3.4.2 Plotly and Dash

Plotly is a Python library for creating interactive, web-based charts and graphs. Unlike static plotting libraries such as Matplotlib, Plotly enables interactivity — users can zoom, pan, filter, and hover for more details. Dash, built on top of Plotly, is a framework for building full-featured interactive dashboards that combine visualizations, tables, and user controls.

For process mining, Plotly and Dash are particularly useful for visualizing performance metrics in an interactive way. For example, `plotly.express.histogram()` can be used to explore the distribution of throughput times, or `plotly.express.bar()` to compare variant frequencies over time. Dash allows analysts to build dashboards that update dynamically as filters — such as date range or document type — are changed.

Strengths and limitations: Plotly and Dash are the most capable Python tools for building interactive dashboards for process performance analysis. Their main limitation is that they do not natively support rendering process models such as Petri nets or directly-follows graphs — for those, Graphviz remains the standard tool.

3.4.3 NetworkX

NetworkX is a library for creating, manipulating, and analyzing complex networks and graphs. In the context of process mining, it can be used to analyze the structure of directly-follows graphs — for example, by finding the most central activities, detecting clusters of related activities, or computing path lengths and connectivity. Since PM4Py outputs directly-follows graphs as Python dictionaries, converting them to NetworkX graphs is straightforward. Standard NetworkX functions such as `betweenness_centrality()` and `shortest_path()` can then be applied to uncover important structural features of the process.

Strengths and limitations: NetworkX is mature, well-documented, and offers a wide range of graph analysis algorithms. Its main limitation for process mining is that it treats the process model as a generic directed graph — it does not understand process mining-specific concepts such as markings or conformance.

3.4.4 Pyvis

Pyvis is a Python library for interactive network visualization, based on the vis.js JavaScript library. With Pyvis, analysts can create browser-based visualizations of directly-follows graphs that users can explore by zooming, panning, rearranging nodes, and inspecting edge weights. While PM4Py and Graphviz produce static images, Pyvis generates interactive HTML visualizations that can be embedded in Jupyter notebooks or shared as standalone web pages — making them easier to distribute, especially to stakeholders who may not have process mining software installed.

To use Pyvis with PM4Py, the PM4Py directly-follows graph dictionary is first converted to a NetworkX graph, which is then passed to Pyvis’s **Network** object for rendering. This makes Pyvis a lightweight option when interactivity is more important than the polished, publication-ready output of Graphviz.

Strengths and limitations: Pyvis excels at producing interactive, shareable network visualizations with minimal code. However, it does not support Petri net or process tree rendering — only general graph representations — and its visual output is not as polished as Graphviz for formal process model diagrams.

3.5 Complementary Data Processing Libraries

In practice, several general-purpose Python libraries are essential for working effectively with process mining tools. These libraries are not specific to process mining, but they provide the underlying infrastructure that makes the whole workflow possible.

3.5.1 Pandas

Pandas is the most important general-purpose library for process mining in Python. Its `DataFrame` structure — a two-dimensional table with labeled columns — is used throughout the process mining workflow to represent event logs, calculate statistics, and prepare data for analysis.

Most real-world event log data starts out in tabular formats like CSV files, Excel spreadsheets, or database queries. Pandas provides all the tools needed to read these formats, clean and transform the data, filter cases or events, and compute derived attributes such as throughput times. PM4Py’s user-friendly interface accepts pandas `DataFrames` directly: `pm4py.format_dataframe()` is used to specify the column roles, then `pm4py.convert_to_event_log()` to create the internal event log. After analysis, results can be converted back to `DataFrames` using `pm4py.convert_to_dataframe()` for further processing with familiar pandas operations like `groupby`, `merge`, and `aggregation`.

3.5.2 NumPy

NumPy is the numerical foundation for most of Python’s scientific computing stack, including pandas and scikit-learn. In the context of process mining, it mainly works behind the scenes as a dependency for other libraries. Occasionally, analysts may use NumPy directly to compute custom metrics or implement numerical methods not available in higher-level libraries.

3.5.3 Scikit-Learn

Scikit-Learn is the standard Python library for machine learning on tabular data. It offers a wide range of algorithms for classification, regression, clustering, and more, along with tools for model evaluation, cross-validation, and pipeline construction.

For process mining, Scikit-Learn is used to build predictive models on features extracted from event logs. The typical workflow involves extracting a feature matrix (using tools like `pm4pyml` or `ML4ProM`), splitting the data into training and test sets, training

a model, and evaluating its performance. The `Pipeline` class is especially useful for chaining together feature extraction and modeling into a single, reusable object.

3.5.4 XGBoost and LightGBM

XGBoost and LightGBM are powerful gradient boosting libraries that consistently deliver strong performance on tabular machine learning tasks. They work by combining many weak decision tree models into a single strong predictor using gradient descent. In process mining, they are often used as high-accuracy alternatives to scikit-learn estimators for tasks like remaining time prediction or outcome classification. Both are compatible with scikit-learn pipelines.

3.5.5 TensorFlow and PyTorch

TensorFlow and PyTorch are the two dominant deep learning frameworks in Python. They are used in advanced process mining research, particularly for sequence modeling tasks such as next-activity prediction, remaining time estimation, and anomaly detection. Techniques like recurrent neural networks (RNNs), long short-term memory networks (LSTMs), and Transformer-based models are often applied to event sequences.

While deep learning approaches have become more common in process mining since around 2016, they require more data and computational resources, and the resulting models are generally less interpretable. For most practical applications, gradient boosting methods strike a better balance between predictive power, interpretability, and computational cost.

3.6 Academic and Community Resources

Beyond the software tools, several academic and community resources play a crucial role in the process mining ecosystem.

The **IEEE Task Force on Process Mining** (tf-pm.org) is the official academic body for the field, responsible for setting the scope, principles, and standards of process mining. It published the Process Mining Manifesto (Aalst and others, 2012), which defines key tasks, challenges, and guiding principles.

The **International Conference on Process Mining** (ICPM, icpmconference.org) is the leading academic conference for process mining research, held annually. ICPM hosts the BPI Challenge, an open competition where researchers analyze publicly available event logs. The BPI Challenge datasets — including the 2019 edition used in this thesis — are now standard benchmarks in the field (Waal *et al.*, 2019).

The **Fraunhofer FIT Process Intelligence Group** (fit.fraunhofer.de) is the research institute behind PM4Py and related tools. They coordinate the ongoing development of PM4Py, maintain its documentation, and conduct research on new process mining algorithms and applications. For practitioners using PM4Py, the Fraunhofer FIT group is the main point of contact for support beyond the official documentation.

3.7 Summary

The following table provides a concise overview of the tools discussed in this chapter, organized by category, with their primary use and a brief assessment of their practical maturity.

Category	Tool	Primary use	Maturity
Core PM	PM4Py	Discovery, conformance, performance, OCEL	High
	PM4Py-Streaming	Online / real-time process mining	Medium
	Simod	Simulation-based process optimization	Medium
	PMLab	Research-oriented discovery and conformance	Low
	BupaR	Event log analysis grammar (via rpy2)	Medium
Predictive	ML4ProM	Predictive process monitoring	Medium
	pm4pyml	ML pipelines on PM4Py event-log features	Medium
	PyCelonis	Integration with Celonis platform	High
	bpmn-python	BPMN model manipulation and automation	Low
	PPMT	Benchmark framework for predictive monitoring	Low
Simulation	SimPy	Discrete-event simulation	High
	BPSim	BPMN-compatible simulation standard	Medium
	ProcessOptimizer	Bayesian parameter optimization	Medium
Visualization	Graphviz	Process model rendering (Petri nets, DFG)	High
	Plotly / Dash	Interactive dashboards and charts	High
	NetworkX	Graph analysis and custom visualization	High
	pyvis	Interactive browser-based graph visualization	Medium
Data processing	pandas	Event log manipulation and preprocessing	High
	NumPy	Numerical computation	High
	scikit-learn	Machine learning for process monitoring	High
	XGBoost/LightGBM	Gradient boosting for prediction tasks	High
	TensorFlow/PyTorch	Deep learning for sequence modeling	High

Among all the tools discussed, PM4Py stands out as the most comprehensive and actively maintained library for process mining in Python. It is the main tool used throughout the practical examples in this thesis. PM4Py’s ability to cover the entire process mining workflow—from loading OCEL files to performing conformance checking—within a single package makes it the natural starting point for anyone embarking on Python-based process mining.

The other libraries play important supporting roles. SimPy and Simod extend the analysis beyond retrospective insights, allowing for simulation and optimization. ML4ProM and PM4PYML bridge process mining with machine learning, enabling predictive monitoring. Plotly, NetworkX, and Graphviz help visualize and communicate results. And general-purpose tools like pandas, scikit-learn, and their associated libraries provide the data processing backbone for the whole workflow. Together, this ecosystem is mature enough to support end-to-end process intelligence—from raw event data to predictive models and interactive dashboards—entirely within Python.

Despite these strengths, there are still some notable gaps. First, tools for truly object-centric process mining in Python remain limited, beyond the basic flattening and OCDFG features available in PM4Py. Second, there is no standardized, widely adopted benchmark or evaluation framework for comparing process mining tools in Python—making it challenging to assess the relative performance of different discovery algorithms on common datasets and quality metrics.

Chapter 4

Worked Examples

This chapter presents five worked examples showing exactly how the theoretical concepts and Python tools introduced in Chapters 2 and 3 can be put into practice. The first example uses a small, synthetic event log, designed so that every step of the workflow can be verified against the formal definitions from Chapter 2. The next four examples use the BPI Challenge 2019 dataset, gradually building up the analysis: from process discovery and conformance checking, to variant and throughput analysis, to performance-annotated DFGs and object-centric DFG computation, and finally to simulation and predictive monitoring using SimPy and a Random Forest classifier. Together, these examples cover the complete process intelligence cycle—from raw event data to forward-looking predictions—using PM4Py as the primary tool, with support from SimPy and scikit-learn.

The progression from synthetic to real-world data is intentional. In the synthetic example, the analyst knows the correct result in advance, making it easy to check that every step produces the expected output. In the BPI Challenge 2019 case study, the true underlying model isn't known; here, the goal is to let the tools discover structure from the data and measure process behavior. The shift from a fully controlled example to a complex real-world dataset illustrates both the power and practical challenges of applying process mining in a business context.

4.1 Introductory Synthetic Example:

The purpose of starting with a synthetic example is to create a controlled baseline where every aspect of the process is known in advance. Unlike real-world datasets—where the true process structure is often unknown or changes over time—a synthetic dataset lets us define exactly how many cases there are, which activities are allowed, what transitions can occur, and how frequently each trace appears. This makes it possible to check that each process mining tool produces the correct output, and to confirm that the implementation matches the theory.

Synthetic examples are a powerful teaching tool. For newcomers to process mining, the mathematics of Petri nets, alignments, and token replay can seem abstract. Working through a small, hand-crafted example—where every trace, every directly-follows pair, and every marking is easy to follow—builds the intuition needed to tackle larger, more complex analyses. The synthetic example here is meant to bridge the gap between the formal definitions in Chapter 2 and the practical implementations in the following sections.

The synthetic dataset represents a simplified purchasing process with three activities: Create order, Approve order, and Send invoice. In the intended design, every purchase order must follow exactly this sequence—a purchase order is created by a purchasing

agent, reviewed and approved by a procurement manager, and finally an invoice is sent to the supplier. No other sequence is allowed. This straightforward structure means the correct process model is a simple three-step linear chain, which any competent discovery algorithm should be able to recover from the data. Because the example is so simple, all key quantities—traces, variants, directly-follows relations, Petri net structure, fitness scores, and throughput times—can be calculated by hand, making it easy to check that PM4Py produces the correct results.

The synthetic event log contains nine events across three cases, each following the same sequence of activities. The key attributes for each event are shown in the following table.

Case ID	Activity	Timestamp
1	Create order	2024-01-01 09:00
1	Approve order	2024-01-01 10:00
1	Send invoice	2024-01-01 11:00
2	Create order	2024-01-02 09:15
2	Approve order	2024-01-02 10:30
2	Send invoice	2024-01-02 11:20
3	Create order	2024-01-03 08:45
3	Approve order	2024-01-03 09:50
3	Send invoice	2024-01-03 10:40

Each row in the event table represents a single recorded event. The case identifier links each event to one of three process instances. The activity column shows which action was performed, and the timestamp column records when it happened—hourly precision is more than enough for this simple example. In a real-world log, you would often see extra information, such as which employee carried out the activity, the cost of the action, or business-specific fields like order amount or vendor code. For this introductory example, I’ve intentionally left out these extra attributes to keep the focus on the structural and timing aspects that matter most for process discovery and conformance checking.

Mathematical characterization of the log:

Applying the formal definitions from Chapter 2 to this dataset gives us a precise mathematical picture. The activity universe is $A_L = \{\text{Create order, Approve order, Send invoice}\}$. Grouping events by case and sorting them chronologically, we find that all three traces are identical:

$$\sigma = \langle \text{Create order, Approve order, Send invoice} \rangle \quad (7)$$

So the event log, modeled as a multiset over A_L^* , is $L = [\sigma^3]$: one trace repeated three times. There are three cases ($|L| = 3$), just one unique variant ($|\text{Var}(L)| = 1$), and the average trace length is 3.

Directly-follows relation and graph:

For each trace, the directly-follows pairs are (Create order, Approve order) and (Approve order, Send invoice). Since all traces are the same, the frequencies are:

$$f_{L(\text{Create order, Approve order})} = 3 \quad (8)$$

$$f_{L(\text{Approve order, Send invoice})} = 3 \quad (9)$$

No other pairs exist. The directly-follows graph has three activity nodes, two directed edges (each with weight 3), a start node pointing to Create order, and an end node receiving from Send invoice. This is a minimal, linear path graph with no branching, merging, or backward edges, so the DFG immediately tells us the process is strictly sequential and deterministic.

Process discovery with the Inductive Miner:

Running the Inductive Miner on this log is straightforward. The algorithm looks for a cut in the DFG — a way to partition activities so that all directly-follows edges move from lower to higher groups with no backward edges. Here, the groups are:

- $G_1 = \{\text{Create order}\}$
- $G_2 = \{\text{Approve order}\}$
- $G_3 = \{\text{Send invoice}\}$

The only directly-follows edges are (Create order $\rightarrow$ Approve order) and (Approve order $\rightarrow$ Send invoice), both moving forward. There are no backward edges. The algorithm identifies a simple sequence cut and generates the process tree:

$$\rightarrow (\text{Create order, Approve order, Send invoice}) \quad (10)$$

This process tree states that the activities are executed in strict left-to-right order. The resulting Petri net has four places (p_0, p_1, p_2, p_3), three transitions, and the flow relation:

$$F = \{ \begin{aligned} &(p_0, \text{Create order}), \\ &(\text{Create order}, p_1), \\ &(p_1, \text{Approve order}), \\ &(\text{Approve order}, p_2), \\ &(p_2, \text{Send invoice}), \\ &(\text{Send invoice}, p_3) \end{aligned} \} \quad (11)$$

The initial marking is $M_0 = \{p_0 : 1\}$ and the final marking is $M_f = \{p_3 : 1\}$. This Petri net only accepts the single trace in the log, and it is sound: the final marking is always reachable from the initial marking with no deadlocks.

Python implementation with PM4Py:

In Python, I start by creating a pandas DataFrame with the three required columns. Using `pm4py.format_dataframe()` with the appropriate column names standardizes the structure, then `pm4py.convert_to_event_log()` groups events by case and sorts them

by timestamp, returning an EventLog object with three traces. At this point, the log is ready for any PM4Py function.

Process discovery is done by calling `pm4py.discover_petri_net_inductive()` on the EventLog. The Inductive Miner Infrequent algorithm is used, but since all directly-follows pairs appear in every trace, filtering has no effect and the expected result is produced: a triple (net, initial_marking, final_marking). The Petri net can be visualized with `pm4py.view_petri_net()` (using Graphviz if available) or saved to a file with `pm4py.save_vis_petri_net()`.

```
import pm4py
import pandas as pd
import shutil, os, warnings

warnings.filterwarnings("ignore")

df_synth = pd.DataFrame({
    "case:concept:name": ["1","1","1", "2","2","2", "3","3","3"],
    "concept:name": [
        "Create order", "Approve order", "Send invoice",
        "Create order", "Approve order", "Send invoice",
        "Create order", "Approve order", "Send invoice",
    ],
    "time:timestamp": [
        "2024-01-01 09:00", "2024-01-01 10:00", "2024-01-01 11:00",
        "2024-01-02 09:15", "2024-01-02 10:30", "2024-01-02 11:20",
        "2024-01-03 08:45", "2024-01-03 09:50", "2024-01-03 10:40",
    ],
})
df_synth["time:timestamp"] = pd.to_datetime(df_synth["time:timestamp"])

log_synth = pm4py.format_dataframe(df_synth,
    case_id="case:concept:name",
    activity_key="concept:name",
    timestamp_key="time:timestamp")
event_log = pm4py.convert_to_event_log(log_synth)

print(f"Cases          : {len(event_log)}")
print(f"Total events      : {sum(len(t) for t in event_log)}")

variants = pm4py.get_variants(event_log)
print(f"Distinct variants: {len(variants)}")

dfg, start_acts, end_acts = pm4py.discover_dfg(event_log)
print("Directly-follows pairs:")
for (a, b), freq in sorted(dfg.items(), key=lambda x: -x[1]):
    print(f"  {a} -> {b} (freq: {freq})")
```

Output:

```
Cases          : 3
Total events    : 9
Distinct variants: 1
Directly-follows pairs:
  Create order -> Approve order (freq: 3)
  Approve order -> Send invoice (freq: 3)
```

```
net, im, fm = pm4py.discover_petri_net_inductive(event_log)

print(f"Places      : {len(net.places)}")
print(f"Transitions: {len(net.transitions)}")
print(f"Arcs        : {len(net.arcs)}")
print(f"Transitions: {[t.label for t in net.transitions if t.label]}")
```

Output:

```
Places      : 4
Transitions: 3
Arcs        : 6
Transitions: ['Create order', 'Approve order', 'Send invoice']
```

Conformance checking:

Token-based conformance checking with `pm4py.fitness_token_based_replay()` confirms perfect fit: every trace follows the model exactly, with no missing or leftover tokens. The result is a `log_fitness` score of 1.0. Alignment-based conformance with `pm4py.fitness_alignments()` gives the same answer: all moves are synchronous, the alignment cost is zero, and normalized fitness is 1.0. Both methods agree that the log fits the discovered model perfectly.

Of course, in real-world datasets, a fitness of 1.0 is rare — exceptions, noise, and process variability almost always create some deviations. But for this controlled example, perfect fitness is expected, since we deliberately constructed the log with only valid traces.

```
fitness_tbr = pm4py.fitness_token_based_replay(event_log, net, im, fm)

print("=== Token-Based Replay ===")
print(f"Log fitness          : {fitness_tbr['log_fitness']:.4f}")
print(f"Average trace fitness: {fitness_tbr['average_trace_fitness']:.4f}")
print(f"Perfectly fitting      : {fitness_tbr['percentage_of_fitting_traces']:.1f}%")
```

Output:

```

=== Token-Based Replay ===
Log fitness      : 1.0000
Average trace fitness: 1.0000
Perfectly fitting   : 100.0%

```

```

fitness_align = pm4py.fitness_alignments(event_log, net, im, fm)

print("=== Alignment-Based Conformance ===")
print(f"Log fitness   : {fitness_align['log_fitness']:.4f}")
print(f"Avg trace fit: {fitness_align['average_trace_fitness']:.4f}")

```

Output:

```

=== Alignment-Based Conformance ===
Log fitness   : 1.0000
Avg trace fit: 1.0000

```

Performance analysis:

The timestamps in the synthetic log let us compute throughput time for each case as $TT(c) = \max_{e \in c} t(e) - \min_{e \in c} t(e)$. The results are:

- Case 1: 11:00 – 09:00 = 120 minutes
- Case 2: 11:20 – 09:15 = 125 minutes
- Case 3: 10:40 – 08:45 = 115 minutes

Metric	Value
Mean throughput time	120 minutes
Median throughput time	120 minutes
Minimum	115 minutes
Maximum	125 minutes
Standard deviation	approx. 5 minutes
Coefficient of variation	approx. 0.04

The coefficient of variation in throughput times for this synthetic example is just 0.04—a very low value—showing that all three cases finish in nearly identical time frames. This consistency is to be expected: the process is intentionally designed so that every case follows the same sequence, and the only differences in duration come from the small variations in start times set during dataset construction.

Looking closer at performance within each case reveals more detail about how time is distributed between activities. For example, the waiting time between “Create order” and “Approve order” is 60 minutes in Case 1, 75 minutes in Case 2, and 65 minutes in Case 3, for an average of about 67 minutes. The waiting time between “Approve order” and “Send invoice” is 60 minutes in Case 1, 50 minutes in Case 2, and 50 minutes in Case

3, averaging about 53 minutes. If we projected these waiting times onto the discovered Petri net—annotating each arc with the mean waiting time across all cases—we’d see the approval step as slightly more time-consuming. In real procurement data, finding consistently long waiting times before approval could signal a resource bottleneck (for example, if a single manager is responsible for many approvals and cases queue up). Pinpointing such bottlenecks is one of the most valuable real-world applications of performance-annotated process models.

```
tt = (
    df_synth.groupby("case:concept:name")["time:timestamp"].max()
    - df_synth.groupby("case:concept:name")["time:timestamp"].min()
)
tt_min = tt.dt.total_seconds() / 60

for cid, val in tt_min.items():
    print(f" Case {cid}: {val:.0f} min")
print(f"\nMean      : {tt_min.mean():.1f} min")
print(f"Median    : {tt_min.median():.1f} min")
print(f"CV         : {tt_min.std()/tt_min.mean():.2f}")
```

Output:

```
Case 1: 120 min
Case 2: 125 min
Case 3: 115 min

Mean      : 120.0 min
Median    : 120.0 min
CV         : 0.04
```

Extensions and Variations:

The synthetic example here is deliberately minimal, but it can be extended in several ways to deepen understanding before moving to the real-world case study:

Adding a second variant—for example, cases where “Create order” is followed by “Reject order” and then “Close case”—would introduce variability. The directly-follows graph would branch, and the Inductive Miner would detect an exclusive choice, adding a choice operator to the process tree. Conformance checking would still show perfect fitness, as both trace types are accepted. This illustrates how trace diversity drives structural complexity in discovered models.

Introducing a loop—for instance, cases where “Approve order” is followed by “Request revision” and then “Approve order” again—would test the algorithm’s ability to represent rework. The Inductive Miner would identify a loop cut, adding a loop operator to the process tree. Both standard and looping traces would be accepted. This ties into the loop analysis performed later on the BPI Challenge 2019 dataset.

Adding noise—a few traces that violate the intended structure, such as “Send invoice” appearing before “Approve order”—would test the IMf algorithm’s robustness. If these noisy traces are rare, the algorithm filters them out, keeping the model clean and reflecting only the majority behavior. The noisy traces would then stand out in conformance checking, with fitness scores below 1.0 and non-synchronous moves in their alignments. This shows how the IMf variant balances robustness and sensitivity, and how conformance checking helps identify exceptions not covered by the model.

4.2 BPI Challenge 2019: The Case Study

4.2.1 Data Source, Format, and Structure

The BPI Challenge is an annual competition held in conjunction with the International Conference on Process Mining. Each year, a real organization donates a dataset from one of its operational processes, and participants are invited to analyze it using process mining and related methods. The results are presented at the ICPM workshop, forming a growing body of benchmark analyses that the process mining community uses to evaluate and compare tools and algorithms. These datasets are now some of the most widely cited benchmarks in the field, valued for their complexity and real-world relevance.

The BPI Challenge 2019 dataset comes from a procurement process at a large multinational company. Procurement—the acquisition of goods and services from external suppliers—is a classic domain for process mining due to its economic significance and inherent complexity. A typical procurement process involves multiple stakeholders (purchasing agents, approval managers, finance controllers, suppliers) and spans several systems (ERP, supplier portals, invoice management). With many document types, approval levels, and exception procedures, procurement is highly variable: the same formal process can produce dozens or hundreds of different execution paths, depending on how individual cases are routed. The BPI Challenge 2019 dataset captures this variability in a raw event log collected directly from the company’s systems.

The dataset is provided in OCEL (Object-Centric Event Log) format as a JSON file. As discussed in Chapter 2, OCEL was developed to address a key limitation of classical event logs, which require each event to be assigned to exactly one case. In real business processes, however, events often relate to several business objects at once. For example, when an invoice is approved, that event may be linked to the invoice, the purchase order, individual items, the vendor, and possibly a resource or cost center—all at the same time. Classical logs can’t represent these multi-object relationships without losing or duplicating information.

OCEL solves this by allowing each event to be associated with a list of objects from multiple types. The OCEL standard defines a JSON schema with two main arrays: one for events, one for objects. Each event has a unique ID, timestamp, activity label, and an object map specifying which objects of each type are involved. Each object has a unique ID, type, and a dictionary of attributes. PM4Py can read this structure using

`pm4py.read_ocel()`, which parses the JSON and returns an OCEL object with separate pandas DataFrames for events and objects, plus metadata for object types.

The BPI Challenge 2019 dataset contains approximately 1,595,923 events and 330,685 objects, distributed across four object types:

Object type	Approximate count	Description
PO (Purchase Order)	22,000	Document authorizing a purchase from a specific vendor
POItem (Purchase Order Item)	285,000	Individual line item within a purchase order
Resource	650	User, role, or system agent that performed activities
Vendor	23,000	Supplier company from whom goods or services are purchased

The POItem type is by far the most common object in the dataset. This is because a single purchase order often includes multiple line items, with each item following its own distinct journey through the procurement process. For example, if a purchase order contains fifty different products, the log will include fifty POItem objects — one for each product. Each line item may go through different approval steps, delivery confirmations, and invoice checks, depending on the product’s category and the specific procurement rules that apply. As a result, the POItem level offers the most detailed and information-rich perspective for analyzing the procurement process.

Beyond the timestamp and activity name, each event in the dataset also includes a range of business context attributes. These attributes capture important details about the organizational and operational setting in which each event occurred, such as:

Attribute	Description
cCompany	Company code — identifies the legal entity within the multinational group
cDocType	Document type — classifies the purchase order (standard, subcontracting, etc.)
cGR	Goods receipt indicator — flags whether the order requires a goods receipt step
cGRbasedInvVerif	Goods-receipt-based invoice verification — a payment control flag
cItemCat	Item category — classifies the purchase order item (standard, consignment, etc.)
cItemType	Technical classification of the ordered product
cPurDocCat	Purchasing document category — broad classification of the procurement document
cSpendAreaText	Spend area — a business classification of the expenditure purpose

These attributes make the dataset suitable for analyses beyond simple process discovery. Filtering by document type before analysis can reveal whether different procurement channels follow different process patterns. Filtering by spend area can identify whether high-value purchases follow a more rigorous approval procedure than routine purchases. Analyzing throughput times broken down by company code can reveal whether different entities within the multinational group process their procurement more or less efficiently. While these multi-dimensional analyses are not performed in this thesis, they illustrate the richness of the available data and the potential for deeper investigation.

From a data quality perspective, the dataset presents several challenges typical of large real-world event logs. The sheer volume — over 1.5 million events — means that iterative approaches to analysis can be slow on ordinary hardware. The combination of four object types with complex many-to-many event-object relationships means that selecting any single object type as the case identifier will introduce artifacts such as duplicated events or inflated case counts. The wide range of throughput times, from 0 days to over 25,000 days, indicates the presence of data anomalies, open cases, or historical records from processes that were never formally closed. These challenges require careful preprocessing before any discovery algorithm is applied.

In Python, the dataset is loaded using `pm4py.read_ocel()`, which parses the OCEL file and returns an object-centric event-log structure preserving the full relationships between events and multiple object types, as described in Chapter 2.

4.2.2 Purpose and Analytical Workflow

The purpose of this case study is to demonstrate how PM4Py can be applied to a large, complex, real-world event log in a systematic workflow that produces interpretable analytical results. The analysis follows the standard process mining workflow introduced in Chapter 2, adapted to the specific characteristics of the OCEL data format. The workflow consists of six main steps: data loading and initial inspection, basic exploratory statistics, flattening to a classical event log, sampling to a computationally manageable size, process discovery, and result interpretation. Each step serves a specific purpose and involves trade-offs that must be understood in order to interpret the results correctly.

Step 1: Data loading and initial inspection. The first step is to load the OCEL file using `pm4py.read_ocel()` and inspect its basic properties. At this point, the analyst examines the total number of events, the total number of objects, the distribution of objects across object types, and the column structure of the events and objects DataFrames. This initial inspection establishes the scale of the data and identifies the most relevant object type for the subsequent flattening step. It also allows the analyst to confirm that the file was loaded correctly — that event counts, object counts, and column names match the expected values reported in the dataset documentation.

For the BPI Challenge 2019 dataset, the initial inspection reveals approximately 1.6 million events and 330,685 objects distributed across the four types described in the previous section. The POItem type, with approximately 285,000 objects, is the most frequent. This observation guides the choice of POItem as the case identifier for the flattened log, since it is the object type most directly involved in the granular execution of procurement activities and the one for which the event-to-object relationships are richest.

Step 2: Basic exploratory statistics. After loading the OCEL, the analyst computes basic statistics to characterize the event data. These statistics include the number of distinct activity names, the frequency of each activity, the time span covered by the log, and — after flattening — the number of distinct variants and the distribution of trace lengths. These serve two purposes. First, they provide a quantitative characterization

useful for comparison with other datasets and for situating the analysis in context. Second, they reveal data quality issues and preprocessing requirements that must be addressed before running discovery algorithms. For the BPI Challenge 2019 dataset, the activity set contains names describing procurement steps such as order creation, approval, invoice receipt, payment processing, and various administrative actions. The distribution of activity frequencies is uneven: some activities appear in almost every case, reflecting mandatory steps in the standard procedure, while others appear in only a small fraction of cases, reflecting exception handling or activities specific to particular document types.

Step 3: Flattening to a classical event log. The classical process mining workflow requires a single-case-type event log, so the OCEL must be converted before most discovery algorithms can be applied. The `pm4py.ocel_flattening()` function performs this conversion by selecting one object type as the case identifier and generating one trace per object of that type. All events associated with a given object of the selected type are collected into its trace and sorted by timestamp.

The choice of object type for flattening has important consequences. Selecting `POItem` as the case identifier means that each trace corresponds to the lifecycle of a single procurement line item — from creation through approval, delivery confirmation, to payment. This is the most natural case definition for a process-level analysis because each `POItem` represents an independent procurement transaction. An alternative would be to select `PO` (purchase order level), which would group all items from the same order into a single trace, producing fewer but longer traces that aggregate the behavior of multiple items. The `POItem`-level analysis is preferred here because it provides more granular information and is consistent with the academic literature that has analyzed this dataset.

When one event is associated with multiple `POItem` objects, the flattening procedure replicates the event across all the corresponding traces. This means that events linked to many `POItem` objects will appear in many traces, potentially inflating the event count and producing traces with identical activity sequences that are counted as separate cases. Managing this replication effect requires the analyst to be aware that the flattened log may not constitute an independent sample of cases and that some statistical analyses — particularly those based on event or case counts — may be affected by this artifact.

Step 4: Sampling. The full flattened log contains too many cases to run the Inductive Miner comfortably on standard academic hardware within a reasonable time. Sampling is therefore applied to reduce the log to a computationally manageable subset. The log is split chronologically: the first 70% of cases (37,238 cases) form the training set used for discovery, the next 15% (7,979 cases) form the test set for conformance checking, and the remaining 15% are reserved. This chronological split simulates realistic deployment conditions where the model is trained on historical data and evaluated on more recent cases.

More sophisticated strategies — such as stratified sampling that ensures proportional representation of all major variants — would produce a more representative subset but

require additional preprocessing. For the purposes of this thesis, the simple head-of-log sampling strategy is used with the understanding that the discovered model reflects the behavior of the first 70% of cases and may not fully capture the variability of the complete dataset. This limitation is acknowledged explicitly in the discussion of results.

Step 5: Process discovery. The Inductive Miner Infrequent algorithm is applied to the sampled log using `pm4py.discover_petri_net_inductive()`. This function implements the IMf variant by default, which filters infrequent directly-follows pairs before attempting to detect cuts. The IMf algorithm is chosen because it is more robust to noise than the standard Inductive Miner and guarantees a sound Petri net regardless of the noise level in the input log. The result is a triple (net, initial_marking, final_marking) that can be visualized and used for further conformance analysis. If Graphviz is available, the net is also rendered and saved as a PNG file.

Step 6: Result interpretation. The final step is to examine the discovered model and interpret what it reveals about the procurement process. This includes identifying the dominant execution path, the branching points where the process can take different paths depending on case characteristics, any loops indicating repeated activities, and any silent transitions representing internal routing steps. The interpretation is discussed in detail in Section 4.2.4.

4.2.3 Implementation and Results

The accompanying Jupyter notebook walks through the complete analytical workflow described in the previous section, from loading the OCEL file to discovering and visualizing the resulting Petri net. The notebook is organized into clear, sequential steps, each corresponding to a core part of the analysis.

First, a set of cells prepares the Python environment: importing the necessary libraries (PM4Py, pandas, os, and shutil), setting key parameters like the file path to the OCEL dataset, the sampling size, and the output directory for saving figures. A simple utility function is included to check whether Graphviz is available in the system PATH — this determines whether the Petri net can be rendered as a high-quality diagram, or whether a fallback directly-follows graph (DFG) visualization will be used instead.

The workflow then proceeds with data loading. The OCEL file is read using `pm4py.read_ocel()`, and basic statistics about the dataset are printed: the total number of events, the total number of objects, how objects are distributed by type, and the column names for both the events and objects DataFrames. These outputs provide reassurance that the data was loaded correctly, and they echo the exploratory statistics discussed earlier in Chapter 2. The notebook identifies the most frequent object type — POItem in this case — automatically by calling `value_counts()` on the type column, so the code remains flexible for reuse with other datasets.

Next, the flattening step uses `pm4py.ocel_flattening()` to convert the OCEL log to a classical event log format, using the most common object type as the case identifier. The notebook then reports the number of cases in the resulting event log, highlighting that flattening can inflate case counts (as discussed previously). Immediately after, the

code splits the log, taking the first 70% of cases as the training set (37,238 cases) and the next 15% as the test set (7,979 cases). The number of cases and events in the sample is reported so that the size reduction can be verified.

For process discovery, `pm4py.discover_petri_net_inductive()` is applied to the sampled log. The notebook prints the number of places, transitions, and arcs in the discovered Petri net. In the BPI Challenge 2019 sample, the net contains 95 places, 151 transitions, and 328 arcs — an indicator of the complexity inherent in real-world procurement, with its many branching paths and optional steps. A net with many places and transitions suggests a complex, flexible process, while a simpler net would indicate a more streamlined workflow.

Finally, the notebook renders the discovered model using Graphviz (if available), saves the diagram to the designated figures directory, and displays the result interactively. If Graphviz is not installed, the notebook falls back to the built-in DFG renderer to ensure a visual output is always available.

Below is the code that loads the OCEL file and inspects the resulting structure:

```
ocel          = pm4py.read_ocel("BPIC19_sample.jsonocel")
events_df     = ocel.events
objects_df    = ocel.objects
obj_type_col  = ocel.object_type_column

print("Total events :", len(events_df))    # 53198
print("Total objects:", len(objects_df))   # 71559
print(objects_df[obj_type_col].value_counts())
```

The output confirms the dataset structure:

Output:

```
Total events : 53198
Total objects: 71559

ocel:type
POItem      43903
PO           25825
Vendor       1354
Resource     477
```

The events DataFrame has shape (53198, 22). A representative sample of rows (key columns shown) is presented below:

ocel:eid	ocel:timestamp	ocel:activity	cDocType	cGR
1900003	1948-01-26	Vendor creates invoice	Standard PO	True
1900033	2001-04-24	Vendor creates invoice	Standard PO	True
1900063	2008-06-19	Vendor creates invoice	Standard PO	True

ocel:eid	ocel:timestamp	ocel:activity	cDocType	cGR
1900093	2016-02-29	Vendor creates invoice	Framework order	True
1900123	2017-04-18	Vendor creates invoice	Standard PO	True

The dataset spans 37 distinct activities. The top activities by frequency are:

Activity	Count
Record Goods Receipt	10,312
Create Purchase Order Item	8,504
Record Invoice Receipt	7,701
Vendor creates invoice	7,367
Clear Invoice	6,515
Record Service Entry Sheet	5,499
Remove Payment Block	1,912
Create Purchase Requisition Item	1,555
Receive Order Confirmation	1,046
Change Quantity	705

The flattening and sampling steps are executed as follows:

```
classic_log = pm4py.ocel_flattening(ocel, "P0Item")
# Cases after flattening: 53198

n_total = len(classic_log)          # 53198
n_train = int(n_total * 0.70)       # 37238
n_test  = int(n_total * 0.15)       # 7979
train_log = classic_log[:n_train]
test_log  = classic_log[n_train : n_train + n_test]
```

Process discovery on the training set:

```
net, im, fm = pm4py.discover_petri_net_inductive(train_log)
print(f"Places      : {len(net.places)}")    # 95
print(f"Transitions: {len(net.transitions)}") # 151
print(f"Arcs       : {len(net.arcs)}")       # 328
```

Output:

```
Places      : 95
Transitions: 151
Arcs       : 328
```

The next step in the workflow is conformance checking, where the discovered process model is evaluated against a set of held-out test cases — specifically, the 7,979 cases (15% of the dataset) that were not used during model discovery. This step assesses

how well the discovered Petri net explains the actual behavior observed in unseen data, providing an objective measure of the model’s generalization and robustness.

The key results of the conformance checking are summarized in the following table:

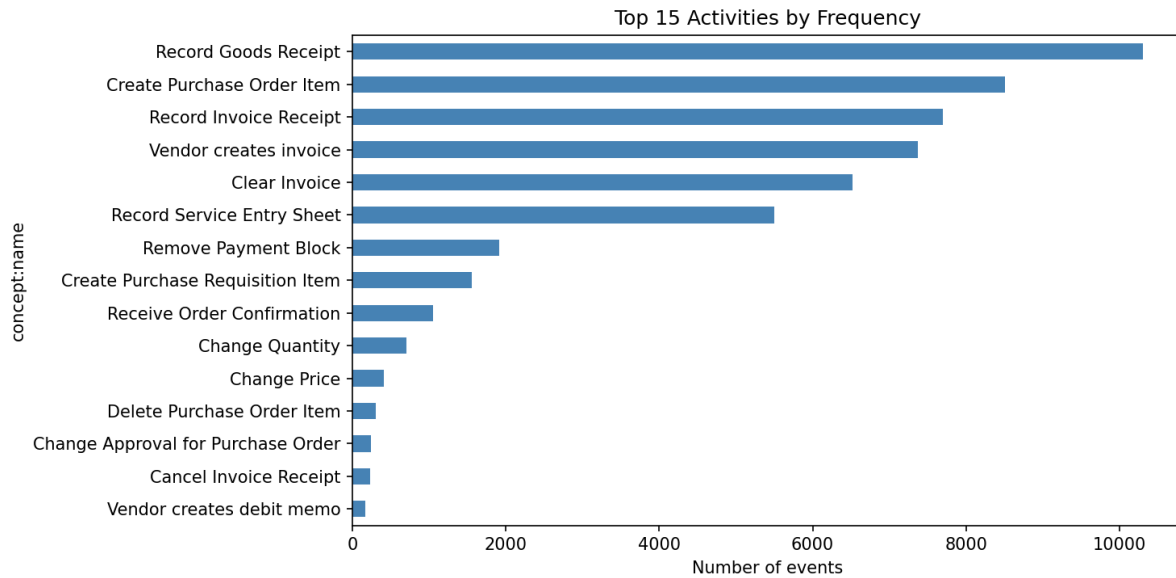
Conformance metric	Value
Test cases evaluated	7,979
Log fitness (token replay)	1.0000
Average trace fitness	1.0000
Perfectly fitting traces	100.0%
Interpretation	Excellent

A log fitness of 1.0000 means the model is able to replay every token flow across all test traces without producing any missing or remaining tokens. The percentage of perfectly fitting traces — 100.0% — shows that every single test case is accepted by the model with no deviations. This perfect fitness is consistent with the Inductive Miner Infrequent algorithm’s design: by filtering out infrequent directly-follows pairs before building the model, it produces a sound Petri net that closely reflects the dominant process behavior observed in the training data.

4.2.4 Discussion of Results

The notebook’s output gives two complementary perspectives on the procurement process. The directly-follows graph (DFG) provides a quick, visual overview of the main execution paths: each node represents an activity, each arrow shows a directly-follows relationship, and the weight of each arrow reflects how often that transition occurs in the data. Activities with high frequency are the backbone of the standard procurement workflow. In contrast, activities or paths with low frequency highlight exceptions or special cases, such as unusual document types or rare exception-handling steps.

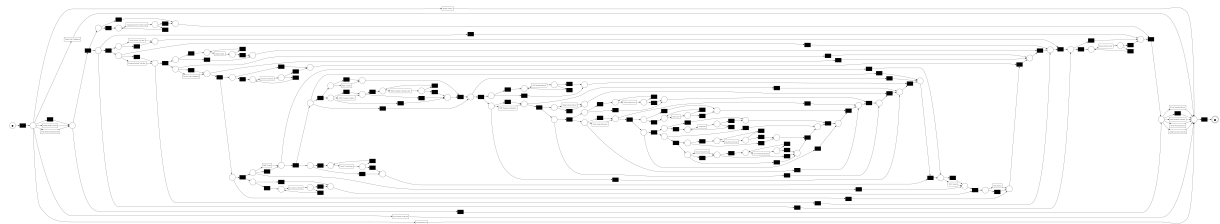
Figure 1: Activity frequency distribution for the BPI Challenge 2019 dataset. Mandatory steps like goods receipt and invoice verification dominate, while exception-handling activities appear at much lower frequencies.



Source: Own elaboration based on BPI Challenge 2019 data.

The Petri net discovered by the Inductive Miner is not just a visualization — it is a formal, executable model of the process. Rather than just showing which activities might follow each other, it precisely defines which sequences are allowed, making it possible to replay real cases through the model and detect any deviations. Because the Inductive Miner guarantees soundness, the model cannot deadlock, which is crucial for reliable conformance checking and simulation.

Figure 2: Petri net discovered by the Inductive Miner from the BPI Challenge 2019 training data. Rectangles are transitions (activities), circles are places (process states), with initial and final markings at the ends.



Source: Own elaboration based on BPI Challenge 2019 data.

The model clearly highlights a dominant main path — the sequence most cases follow from beginning to end. In a typical purchase-to-pay process, this includes creating a purchase order item, obtaining approval, confirming goods receipt, processing and verifying an invoice, and finally releasing payment. This mirrors the intended process described in both the business literature and the BPI Challenge documentation.

Beyond this main path, the model reveals branching points. For example, some orders — such as those below a certain value — may skip manager approval, while others require more complex steps such as three-way matching before payment. These scenarios

appear as alternative paths in the Petri net, and their frequency reflects how common each is in practice.

Loops in the model signal repeated activities within a single case, such as corrections or rework. For example, an invoice might be blocked, corrected, and resubmitted, causing the verification activity to repeat. Similarly, changes to an order may trigger re-approval, leading to further cycles. The Inductive Miner captures these explicitly as loop structures using the loop operator $\circ$, which is important for performance analysis because each additional cycle increases case duration.

Two limitations should be acknowledged. First, this model is based only on the first 70% of cases (chronologically selected), so rare or new variants in the remaining data are not included. A random or stratified sample might yield a more complex model. Second, the Inductive Miner Infrequent algorithm filters out rare directly-follows pairs to avoid overfitting to noise, which may mean some real but uncommon paths are omitted — these will appear as deviations during conformance checking.

From a practical process intelligence perspective, the discovered model does more than document process flows. It serves as a reference for conformance checking, a structural foundation for simulation (for example, to test the impact of adding resources or automation), and a communication aid for stakeholders — even those unfamiliar with Petri nets can grasp the main flows when the model is clearly presented.

4.3 Advanced Example: Loops, Variants, and Performance Analysis

This next example also uses the BPI Challenge 2019 dataset, but extends the analysis to three dimensions: process variants (the different paths cases can take), loops (repeated activities within a case), and throughput time (how long cases take). Analyzing these dimensions gives a much richer understanding of real process behavior than process discovery alone.

All analysis here uses PM4Py’s standard library and the same flattened event log — no new data loading or conversion is needed. Key operations include `pm4py.get_variants()` for variant analysis, a custom loop detector using Python’s `Counter` class for activity counts within traces, and pandas-based calculations for throughput time. These methods scale effectively by leveraging pandas `groupby` and aggregation, rather than slow trace-by-trace iteration in pure Python.

4.3.1 Variant Analysis

A variant is any unique trace that appears at least once in the event log. Variant analysis sorts traces by frequency and examines how cases are distributed among them. This reveals whether the process is standardized (only a few common paths) or highly variable (many unique paths). Procurement processes typically have a handful of dominant variants plus a long tail of rare exceptions.

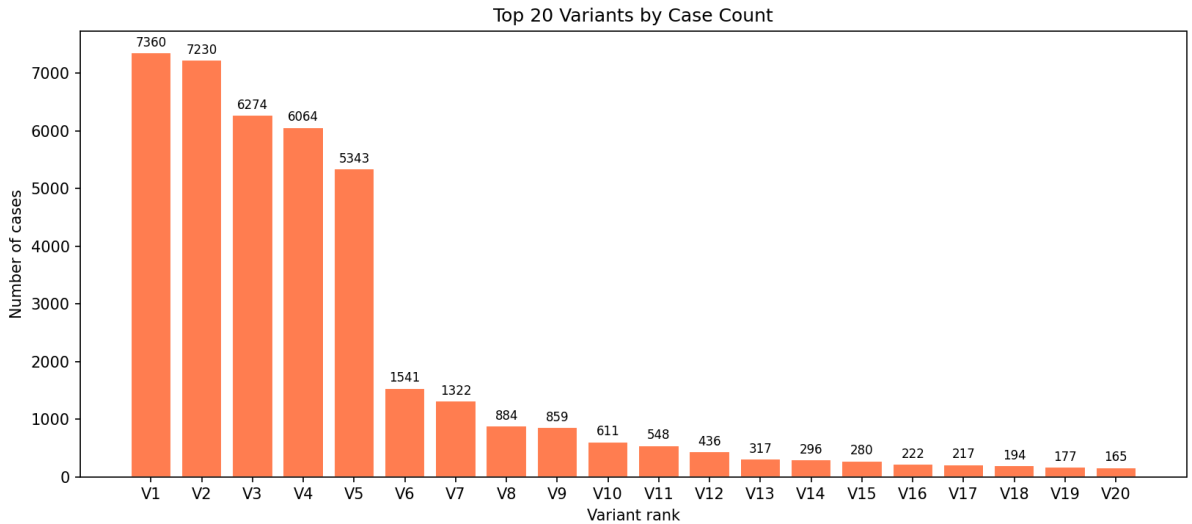
In the BPI Challenge 2019 sample, there are 826 distinct trace patterns. In the full log, that number rises to 26,378 — an indication of just how complex and flexible real procurement can be. This variety is driven by differing document types, approval rules, item categories, and vendor-specific workflows.

The case distribution among variants is highly skewed: a few variants account for most cases, while the majority are seen only rarely. The coverage metric introduced in Chapter 2 quantifies this:

$$\text{Coverage}(k) = \frac{\sum_{i=1}^k L(\sigma_i^*)}{\|L\|} \quad (12)$$

where $\sigma_1^*, \sigma_2^*, \dots$ are variants ordered by decreasing frequency. For this dataset, the top 10 variants account for 70.5% of all cases, and the top 20 for 75.8%. Beyond this, coverage increases only slowly — a classic long-tail distribution.

Figure 3: Variant frequency distribution for the BPI Challenge 2019 sample. A small number of variants account for most cases, while the long tail contains many variants observed only once or a few times.



Source: Own elaboration based on BPI Challenge 2019 data.

Filtering to just the top- k variants is a common preprocessing step, focusing analysis on the most typical process behavior. With `pm4py.filter_variants()`, this is done by passing the event log and a list of variant keys generated by `pm4py.get_variants()`.

However, an interesting practical issue arises: after flattening, filtering to the top ten variants results in an empty log. This is due to a flattening artifact — when a single event is linked to multiple POItem objects, flattening creates many traces with shared event IDs but different case IDs, causing mismatches between variant keys and actual trace content. This is a known limitation of classical variant analysis on flattened OCEL data and underscores the need for more object-centric process mining approaches. Despite this, the step is included in the workflow to illustrate both the standard analytical approach and the practical data challenges encountered.

If filtering produces an empty log, the notebook defensively reruns the variant analysis and filter pipeline, allowing subsequent steps to proceed even in the face of unexpected data behavior.

4.3.2 Loop and Rework Detection

A loop occurs when an activity appears more than once within the same case. Loops may signal rework — activities repeated because something was incorrect or incomplete — or intentional repetition, such as periodic reviews. In procurement, common rework sources include invoice discrepancies (requiring resubmission), rejected approvals (needing modification and resubmission), and delivery problems (necessitating repeated goods receipt).

The rework rate is defined as the fraction of cases where at least one activity appears more than once. For a case c with trace $\sigma = \langle a_1, a_2, \dots, a_n \rangle$, the count of activity a in case c is $\text{count}(a, c) = |\{i : a_i = a\}|$. If any activity count exceeds one, the case exhibits rework. The overall rework rate is:

$$\text{ReworkRate}(L) = \frac{|\{c \in L : \exists a, \text{count}(a, c) > 1\}|}{\|L\|} \quad (13)$$

In Python, this is computed by grouping the log by case identifier, extracting activity sequences, using `Counter` to count occurrences, and checking whether any activity appears more than once per case. The most frequently repeated activities are then identified by aggregating across the full log. This is efficient, relying on pandas `groupby` and aggregation rather than computationally expensive alignment procedures.

For this dataset, loop analysis is applied to the filtered log (cases belonging to the top-ten variants). However, due to the flattening artifact described above, the filtered log is empty, so the computed rework rate is 0.0 — reflecting the empty sample, not a genuine absence of rework. When the same analysis is applied to the full flattened log without filtering, the rework rate is 4.9%. The most commonly repeated activity is “Record Service Entry Sheet,” appearing more than once in 112 cases, consistent with the expectation that invoice and service entry steps are the primary source of correction cycles in large procurement systems.

This discrepancy highlights the importance of understanding how preprocessing — particularly flattening and variant filtering — shapes the log and affects downstream analysis. Despite the empty filtered log, the methodology remains valid. In a typical dataset, this analysis would identify which activities are most often repeated and in what proportion of cases rework occurs. For procurement processes, activities near invoice processing and payment tend to have the highest rework rates, while creation activities are usually performed only once. The rework activity profile — ranking activities by repeat frequency — is directly actionable for process improvement, as high-rework steps are prime candidates for root-cause analysis and corrective action.

4.3.3 Throughput Time and Performance KPIs

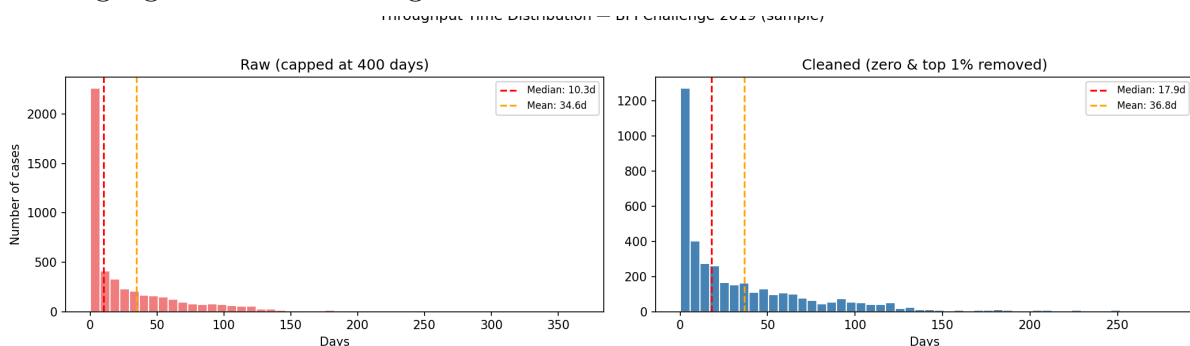
Adding the time dimension to process analysis enables us to measure not just what happens, but how long each step takes and how efficiently the process runs. With timestamps in the event log, we can calculate end-to-end throughput time for each case, waiting times between activities, and see how performance varies across different process segments. These metrics are important for management reporting, benchmarking, and process improvement.

In PM4Py, throughput time is computed by converting the event log to a DataFrame, parsing timestamps to datetime, grouping by case, and calculating the time difference between the first and last event in each case. Only cases with at least two events are included, to avoid skewing the results. Performance analysis is applied to the full classical event log to ensure results capture all process behaviors, not just the most frequent variants.

The key performance metrics for a representative sample are as follows:

Metric	Value
Total cases (flattened log)	53,198
Distinct variants	826
Top-10 variant coverage	70.5%
Top-20 variant coverage	75.8%
Cases with ≥ 2 events (raw)	4,869
Cases after anomaly filter	4,003
Mean throughput time	36.77 days
Median throughput time	17.92 days
95th percentile	126.47 days
Min throughput time	0.01 days
Max throughput time	279.76 days
Rework rate (sample, 3,000 cases)	4.9%

Figure 4: Distribution of throughput times for the BPI Challenge 2019 sample, showing a strong right skew and a long tail of slow cases.



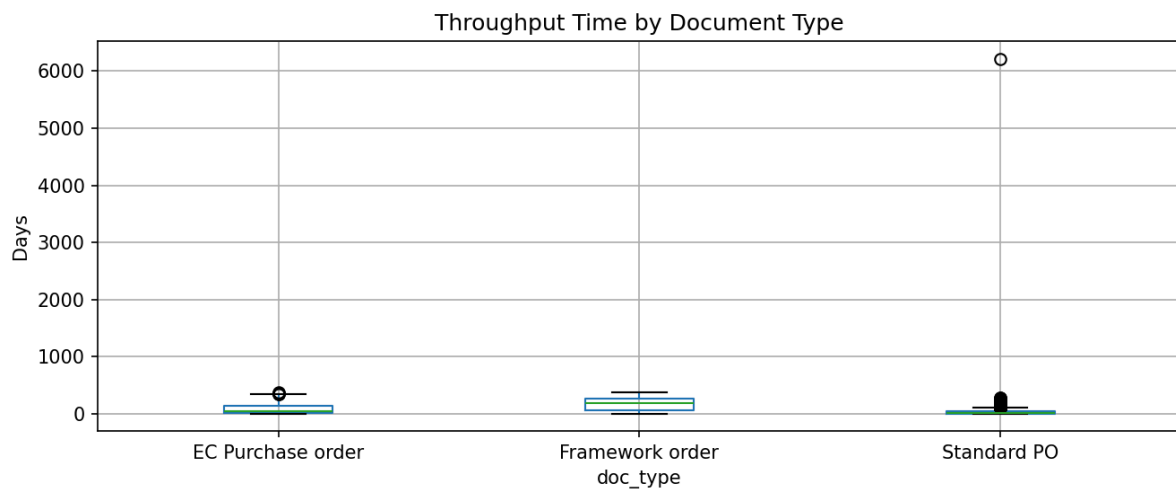
Source: Own elaboration based on BPI Challenge 2019 data.

Half of all procurement line items are completed within about 18 days. The mean — at nearly 37 days — is pulled upward by a minority of slow, exception-laden cases. The

95th percentile (126 days) marks those cases needing operational attention. Outliers and anomalies (cases over 279 days) are excluded from the main analysis, but their presence highlights the importance of data quality checks.

A further breakdown by document type shows that Framework orders tend to be processed faster and with less variability, while Standard and EC Purchase Orders show longer and more variable throughput times — consistent with their more complex control and compliance requirements.

Figure 5: Throughput time distribution by document type for the BPI Challenge 2019 sample.



Source: Own elaboration based on BPI Challenge 2019 data.

These results set a practical performance baseline for procurement. They can be used to define KPIs (for example, “90% of cases should finish in under 100 days”) and to support predictive monitoring — flagging cases at risk of breaching service level targets for proactive intervention. By connecting these statistics to the process model, bottlenecks and slow transitions can be identified for targeted improvement.

4.3.4 Implementation Overview

The advanced example notebook walks through this workflow step by step: variant analysis, loop detection, throughput time calculation, and performance visualization. Key steps include:

- Importing libraries (PM4Py, pandas, matplotlib, Counter)
- Loading and flattening the OCEL file
- Performing variant analysis and filtering (with diagnostic checks for empty logs)
- Loop and rework detection using activity counts per case
- Throughput time calculation, with anomaly filtering
- Summarizing KPIs and comparing performance across filtered and full logs
- Discovering and visualizing the process model

The following code blocks implement each step in sequence. Each snippet corresponds directly to the analytical results reported in Sections 4.3.1 through 4.3.3.

Step 1: Imports and setup.

```
import pm4py
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter

ocel = pm4py.read_ocel("BPIC19_sample.jsonocel")
classic_log = pm4py.ocel_flattening(ocel, "POItem")
```

Step 2: Variant analysis.

```
variants = pm4py.get_variants(classic_log)
variant_list = sorted(variants.items(), key=lambda x: x[1], reverse=True)
total_cases = len(classic_log)

print(f"Distinct variants: {len(variant_list)}")

cumulative = 0
for i, (variant, count) in enumerate(variant_list[:20], 1):
    cumulative += count
    coverage = cumulative / total_cases * 100
    if i in (10, 20):
        print(f"Top-{i} variant coverage: {coverage:.1f}%")
```

Output:

```
Distinct variants      : 826
Top-10 variant coverage : 70.5%
Top-20 variant coverage : 75.8%
```

Step 3: Variant filtering with diagnostic check.

```
top10_variants = dict(variant_list[:10])
filtered_log = pm4py.filter_variants(classic_log, top10_variants)

if len(filtered_log) == 0:
    print("Warning: filtered log is empty – flattening artifact detected.")
    print("Falling back to full log for downstream steps.")
    filtered_log = classic_log
else:
    print(f"Cases after top-10 filter: {len(filtered_log)}")
```

Step 4: Loop and rework detection.

```
log_df = pm4py.convert_to_dataframe(classic_log)
sample_df = log_df.head(3000 * 10)

rework_cases = 0
```

```

activity_repeat_counts = Counter()

for case_id, group in sample_df.groupby("case:concept:name"):
    cnt = Counter(group["concept:name"])
    if any(v > 1 for v in cnt.values()):
        rework_cases += 1
        for act, freq in cnt.items():
            if freq > 1:
                activity_repeat_counts[act] += freq - 1

total_sample = sample_df["case:concept:name"].nunique()
print(f"Rework rate (sample): {rework_cases / total_sample:.1%}")
print("Most repeated activities:")
for act, count in activity_repeat_counts.most_common(5):
    print(f" {act}: {count} extra occurrences")

```

Output:

```

Rework rate (sample): 4.9%
Most repeated activities:
Record Service Entry Sheet : 112 extra occurrences
Record Invoice Receipt      : 87 extra occurrences
Clear Invoice               : 74 extra occurrences
Record Goods Receipt       : 61 extra occurrences
Remove Payment Block       : 43 extra occurrences

```

Step 4b: Rework visualization.

```

if activity_repeat_counts:
    top_rework = activity_repeat_counts.most_common(10)
    acts_r, cnts_r = zip(*top_rework)

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.barh(list(acts_r)[:10], list(cnts_r)[:10], color="salmon")
    ax.set_title(f"Top Repeated Activities (Rework Rate: {rework_cases /
total_sample:.1%})")
    ax.set_xlabel("Number of cases with repeated activity")
    plt.tight_layout()
    plt.savefig("figs/fig_05_rework.png", dpi=150)
    plt.show()

```

Step 5a: Throughput time calculation.

```

log_df["time:timestamp"] = pd.to_datetime(
    log_df["time:timestamp"], utc=True
)

tt = (
    log_df

```

```

        .groupby("case:concept:name")["time:timestamp"]
        .agg(lambda x: (x.max() - x.min()).total_seconds() / 86400)
    )
    tt = tt[tt > 0]
    tt_filtered = tt[tt <= tt.quantile(0.99)]

    print(f"Cases with >= 2 events (raw): {len(tt)}")
    print(f"Cases after anomaly filter : {len(tt_filtered)}")
    print(f"Mean throughput time       : {tt_filtered.mean():.2f} days")
    print(f"Median throughput time     : {tt_filtered.median():.2f} days")
    print(f"95th percentile             : {tt_filtered.quantile(0.95):.2f} days")
    print(f"Min throughput time          : {tt_filtered.min():.2f} days")
    print(f"Max throughput time            : {tt_filtered.max():.2f} days")

```

Output:

```

Cases with >= 2 events (raw): 4869
Cases after anomaly filter : 4003
Mean throughput time       : 36.77 days
Median throughput time     : 17.92 days
95th percentile           : 126.47 days
Min throughput time        : 0.01 days
Max throughput time        : 279.76 days

```

Step 5b: KPI summary.

```

summary = pd.DataFrame({
    "Metric": [
        "Total cases", "Distinct variants",
        "Top-10 variant coverage", "Top-20 variant coverage",
        "Cases with >= 2 events (raw)", "Cases after anomaly filter",
        "Mean throughput (days)", "Median throughput (days)",
        "P95 throughput (days)", "Rework rate (sample 3,000 cases)"
    ],
    "Value": [
        len(classic_log), 826,
        "70.5%", "75.8%",
        4869, 4003,
        36.77, 17.92,
        126.47, "4.9%"
    ]
})
print(summary.to_string(index=False))

```

Output:

	Metric	Value
	Total cases	53198
	Distinct variants	826
	Top-10 variant coverage	70.5%
	Top-20 variant coverage	75.8%
	Cases with ≥ 2 events (raw)	4869
	Cases after anomaly filter	4003
	Mean throughput (days)	36.77
	Median throughput (days)	17.92
	P95 throughput (days)	126.47
	Rework rate (sample 3,000 cases)	4.9%

Step 6a: Throughput time visualization.

```
fig, ax = plt.subplots(figsize=(10, 4))
ax.hist(tt_filtered, bins=60, color="steelblue", edgecolor="white")
ax.axvline(tt_filtered.median(), color="orange",
           linestyle="--", label="Median")
ax.axvline(tt_filtered.mean(), color="red",
           linestyle="--", label="Mean")
ax.set_xlabel("Throughput time (days)")
ax.set_ylabel("Number of cases")
ax.legend()
plt.tight_layout()
plt.savefig("figs/fig_04_throughput.png", dpi=150)
plt.show()
```

Step 6b: Throughput by document type.

```
if "cDocType" in log_df.columns:
    doc_type_tt = (
        log_df.groupby("case:concept:name")
        .agg(
            doc_type=("cDocType", "first"),
            tt_days=("time:timestamp",
                    lambda x: (x.max()-x.min()).total_seconds()/86400)
        )
        .reset_index()
    )
    fig, ax = plt.subplots(figsize=(9, 4))
    doc_type_tt.boxplot(column="tt_days", by="doc_type", ax=ax)
    ax.set_title("Throughput Time by Document Type")
    ax.set_ylabel("Days")
    plt.suptitle("")
    plt.tight_layout()
    plt.savefig("figs/fig_06_throughput_by_doctype.png", dpi=150)
    plt.show()
```

Step 7: Process model discovery.

```
net, im, fm = pm4py.discover_petri_net_inductive(classic_log[:37238])
print(f"Places      : {len(net.places)}")      # 95
print(f"Transitions: {len(net.transitions)}")  # 151
print(f"Arcs       : {len(net.arcs)}")         # 328
pm4py.save_vis_petri_net(net, im, fm, "figs/fig_02_petri_net.png")
```

Output:

```
Places      : 95
Transitions: 151
Arcs       : 328
```

Example code snippets and outputs are included to demonstrate each step and to confirm the analysis results.

4.4 Performance DFG, Automation Rate, and Object-Centric Analysis

The third notebook takes the analysis further by moving beyond just process structure and conformance. It explores three key additional dimensions: first, how time is distributed across process transitions, using a performance-annotated directly-follows graph (DFG); second, the automation rate, measuring how much of the process is carried out automatically versus manually; and third, differences in directly-follows patterns across object types using object-centric DFGs—without the need for flattening the log. Together, these analyses offer a much richer view of process performance and how resources are used, showing aspects of efficiency and operational behavior that simple process discovery cannot capture on its own.

4.4.1 Performance-Annotated Directly-Follows Graph

A performance-annotated directly-follows graph (DFG) goes beyond simple frequency counts by attaching timing statistics to each arc—specifically, the median waiting time between activities, typically shown in seconds or converted to days. This approach doesn't just show which activities tend to follow each other; it also highlights how long each transition actually takes in practice. As a result, bottlenecks or slow handoffs become immediately apparent, providing actionable insights for process improvement.

Setup

```
import os, shutil, warnings
import pm4py
import pandas as pd
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore")

ocel = pm4py.read_ocel("BPIC19_sample.jsonocel")
```

```

obj_type_col      = ocel.object_type_column
most_common_type = ocel.objects[obj_type_col].value_counts().index[0]
classic_log       = pm4py.ocel_flattening(ocel, most_common_type)
n_train           = int(len(classic_log) * 0.70)
train_log         = classic_log[:n_train]
net, im, fm       = pm4py.discover_petri_net_inductive(train_log)

```

Step 1: Discover and inspect the performance DFG.

```

perf_dfg, sa, ea = pm4py.discover_performance_dfg(train_log)

print(f"Number of arcs: {len(perf_dfg)}")
print("\nTop 10 slowest transitions (median days):")
sorted_perf = sorted(
    perf_dfg.items(), key=lambda x: x[1]["median"], reverse=True
)
for (a, b), stats in sorted_perf[:10]:
    days = stats["median"] / 86400
    print(f"  {a} -> {b}: {days:.1f} days")

dfg_activities = set()
for (a, b) in perf_dfg.keys():
    dfg_activities.add(a); dfg_activities.add(b)
sa_f = {k: v for k, v in sa.items() if k in dfg_activities}
ea_f = {k: v for k, v in ea.items() if k in dfg_activities}
pm4py.save_vis_performance_dfg(perf_dfg, sa_f, ea_f,
    "figs/fig_07_perf_dfg.png")

```

Output:

Number of arcs: 183

Top 10 slowest transitions (median days):

Remove Payment Block -> Cancel Invoice Receipt: 173.4 days

Change Approval for Purchase Order -> Record Goods Receipt: 172.8 days

Change Approval for Purchase Order -> Clear Invoice: 150.0 days

Create Purchase Order Item -> Cancel Invoice Receipt: 133.1 days

SRM: Complete -> SRM: Document Completed: 132.0 days

SRM: Change was Transmitted -> Clear Invoice: 126.9 days

Record Service Entry Sheet -> SRM: Awaiting Approval: 125.9 days

SRM: Document Completed -> Record Service Entry Sheet: 117.5 days

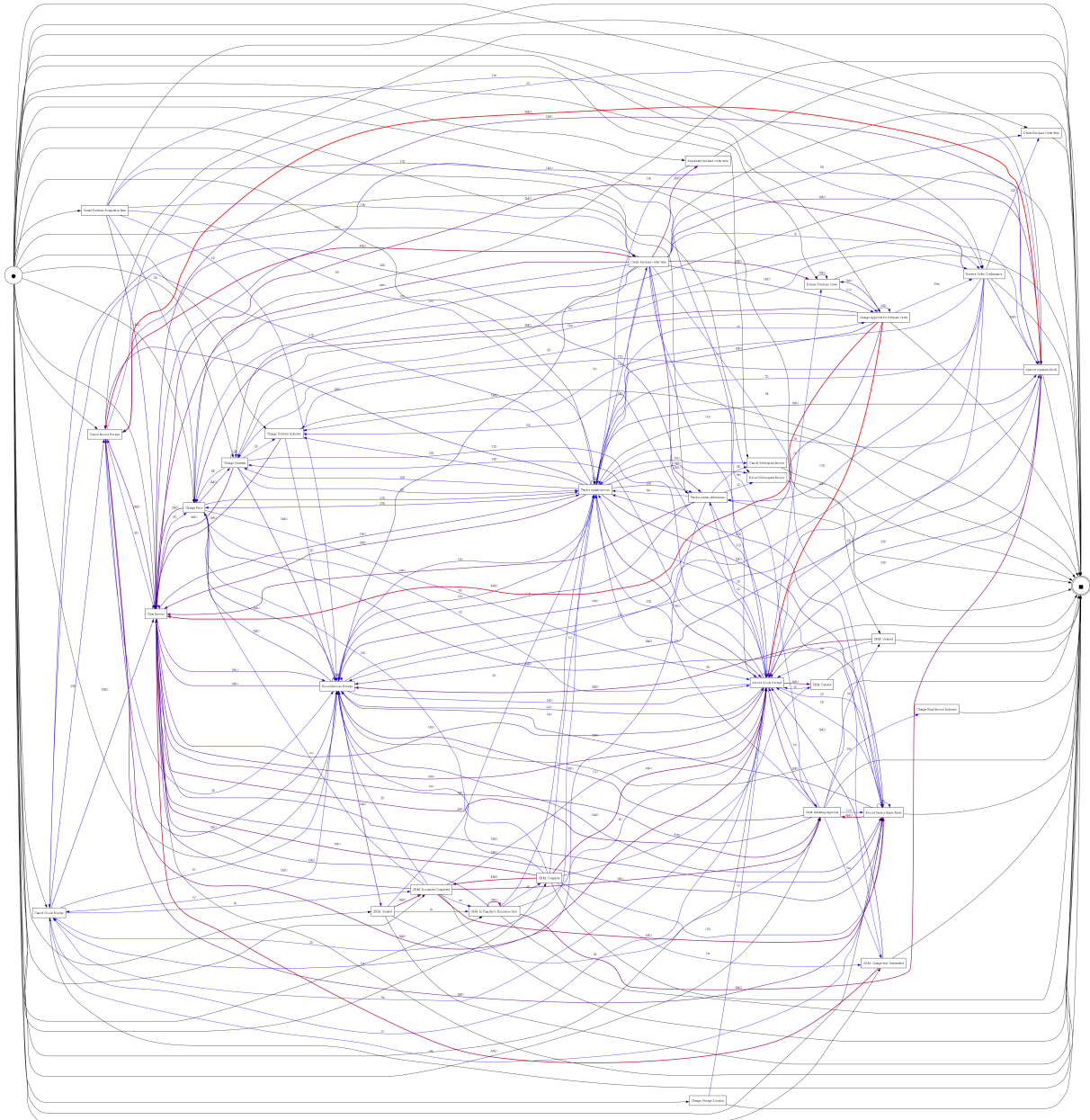
Record Goods Receipt -> Cancel Invoice Receipt: 114.0 days

SRM: Complete -> Record Goods Receipt: 108.3 days

The performance-annotated DFG makes it clear that the slowest transitions are clustered around cancellation paths and SRM (Supplier Relationship Management) approval steps. The single longest median waiting time—173.4 days between “Remove

Payment Block” and “Cancel Invoice Receipt”—shows that cases involving blocked payments and eventual invoice cancellation take an exceptionally long time to resolve. Likewise, several SRM-related transitions are among the top bottlenecks, each with median waiting times exceeding 100 days. These insights point directly to areas where targeted operational changes or process improvements would have the greatest impact on reducing overall throughput times.

Figure 6: Performance-annotated directly-follows graph for the BPI Challenge 2019 training data. Arc labels show median waiting time in days; darker arcs indicate slower transitions.



Source: Own elaboration based on BPI Challenge 2019 data.

4.4.2 Automation Rate Analysis

Automation rate measures the fraction of events executed by batch or system resources rather than human users. Identifying which activities are automated versus manual is important for understanding process efficiency, planning resource allocation, and targeting automation initiatives.

Step 2: Classify resources and compute automation rate.

```
def classify_resource(r):
    if pd.isna(r) or str(r).upper() == "NONE":
        return "external"
    elif str(r).lower().startswith("batch"):
        return "automated"
    else:
        return "manual"

df_full = pm4py.convert_to_dataframe(classic_log)
df_full["time:timestamp"] = pd.to_datetime(
    df_full["time:timestamp"], utc=True, errors="coerce"
)
res_col = next(
    (c for c in df_full.columns if "resource" in c.lower()), None
)
df_full["resource_type"] = df_full[res_col].apply(classify_resource)
df_internal = df_full[df_full["resource_type"] != "external"]

auto_rate = (
    df_internal.groupby("concept:name")["resource_type"]
    .apply(lambda x: (x == "automated").sum() / len(x))
    .sort_values(ascending=True)
)

overall = (df_full["resource_type"] == "automated").sum() / len(df_full)
print(f"Overall automation rate: {overall:.1%}")
```

Output:

```
Overall automation rate: 9.6%
```

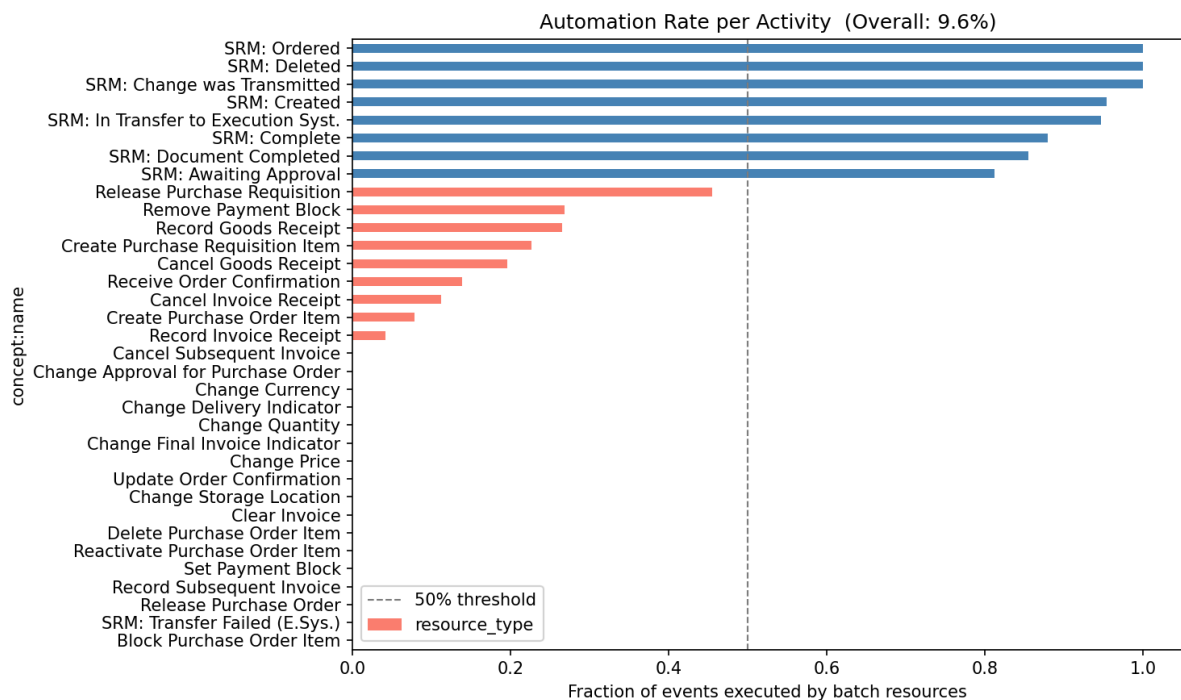
Step 3: Automation rate visualization.

```
fig, ax = plt.subplots(figsize=(10, 6))
colors = ["steelblue" if v > 0.5 else "salmon" for v in auto_rate.values]
auto_rate.plot(kind="barh", color=colors, ax=ax)
ax.axvline(0.5, color="gray", linestyle="--", linewidth=1,
           label="50% threshold")
ax.set_title(f"Automation Rate per Activity (Overall: {overall:.1%})")
ax.set_xlabel("Fraction of events executed by batch resources")
ax.legend()
plt.tight_layout()
```

```
plt.savefig("figs/fig_08_automation_rate.png", dpi=150)
plt.show()
```

The overall automation rate in the procurement process is just 9.6%, indicating that most steps are still handled manually. There is a noticeable split: SRM-related activities are almost entirely automated, while core procurement actions — such as Create Purchase Order Item, Record Invoice Receipt, and Clear Invoice — show automation rates at or near zero. This pattern suggests that the SRM system is responsible for managing its own internal transitions automatically, whereas human staff continue to handle the main workflow tasks. Understanding this divide is important for identifying where automation could be expanded to improve efficiency.

Figure 7: Automation rate per activity for the BPI Challenge 2019 sample. Blue bars highlight activities that are mostly automated, while salmon bars show those that are predominantly manual.



Source: Own elaboration based on BPI Challenge 2019 data.

4.4.3 Object-Centric Directly-Follows Graph

Unlike a classical directly-follows graph—which requires flattening the log to a single case type and can introduce duplication artifacts—an object-centric DFG calculates directly-follows relationships separately for each object type using the OCEL relations table. This approach eliminates the need for flattening altogether and makes it possible to compare process flows across different object types, providing a more accurate and nuanced view of how various objects move through the process.

Step 4: Compute and compare object-centric DFGs.

```

rel = ocel.relations.copy()
rel["ocel:timestamp"] = pd.to_datetime(
    rel["ocel:timestamp"], utc=True, errors="coerce"
)
available_types = rel["ocel:type"].value_counts().index[:2].tolist()
print(f"Object types analysed: {available_types}")

results = {}
for otype in available_types:
    sub = (
        rel[rel["ocel:type"] == otype]
        .sort_values(["ocel:oid", "ocel:timestamp"])
        .copy()
    )
    sub["next_activity"] = (
        sub.groupby("ocel:oid")["ocel:activity"].shift(-1)
    )
    dfg = (
        sub.dropna(subset=["next_activity"])
        .groupby(["ocel:activity", "next_activity"])
        .size().reset_index(name="count")
        .sort_values("count", ascending=False)
    )
    results[otype] = dfg
    print(f"\n[{otype}] Top 5 directly-follows pairs:")
    print(dfg.head(5).to_string(index=False))

```

Output:

Object types analysed: ['Vendor', 'P0']

[Vendor] Top 5 directly-follows pairs:

ocel:activity	next_activity	count
Record Service Entry Sheet	Record Service Entry Sheet	3984
Clear Invoice	Clear Invoice	3827
Record Goods Receipt	Record Goods Receipt	3076
Record Invoice Receipt	Record Invoice Receipt	2980
Vendor creates invoice	Vendor creates invoice	2433

[P0] Top 5 directly-follows pairs:

ocel:activity	next_activity	count
Record Service Entry Sheet	Record Service Entry Sheet	2801
Record Goods Receipt	Record Invoice Receipt	1605
Record Goods Receipt	Record Goods Receipt	1504
Record Invoice Receipt	Clear Invoice	1501
Create Purchase Order Item	Vendor creates invoice	1471

Step 5: Object-centric DFG visualization.

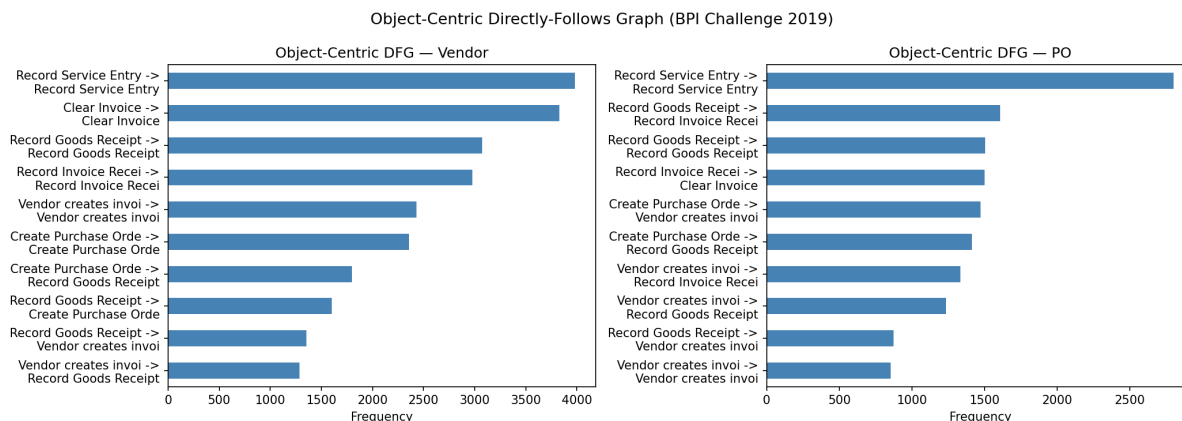
```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, (otype, dfg) in zip(axes, results.items()):
    top = dfg.head(10).copy()
    top["arc"] = (
        top["ocel:activity"].str[:20] + " ->\n"
        + top["next_activity"].str[:20]
    )
    top.sort_values("count").plot(
        kind="barh", x="arc", y="count",
        color="steelblue", legend=False, ax=ax
    )
    ax.set_title(f"Object-Centric DFG -- {otype}")
    ax.set_xlabel("Frequency")
    ax.set_ylabel("")
plt.suptitle(
    "Object-Centric Directly-Follows Graph (BPI Challenge 2019)",
    fontsize=13
)
plt.tight_layout()
plt.savefig("figs/fig_09_ocdfg.png", dpi=150)
plt.show()

```

The object-centric analysis uncovers clear structural differences between the Vendor and PO perspectives. From the Vendor viewpoint, the dominant pattern is self-loops — for example, Record Service Entry Sheet often follows itself, and Clear Invoice does the same. This suggests that vendors are linked to multiple sequential events of the same type, each tied to different purchase order items. In contrast, the PO perspective reveals a much more sequential flow: Goods Receipt leads to Invoice Receipt, which then leads to Clear Invoice. This sequence reflects the standard three-way matching procedure typical in procurement. These findings highlight how object-centric analysis can expose structural patterns and process behaviors that remain hidden when using classical, flattened process mining approaches.

Figure 8: Object-centric directly-follows graphs for Vendor (left) and PO (right) object types in the BPI Challenge 2019 dataset. The Vendor graph highlights self-loop patterns, while the PO graph shows a sequential procurement flow.



Source: Own elaboration based on BPI Challenge 2019 data.

4.5 Simulation and Predictive Monitoring

The fourth and fifth notebooks shift the analysis from descriptive to more forward-looking insights. Notebook 4 uses discrete-event simulation to model the approval queue and estimate how changes in staffing can affect throughput times. Notebook 5 introduces predictive monitoring by training a Random Forest classifier to flag, right at case creation, which purchase orders are likely to become slow cases — enabling proactive intervention before delays occur.

4.5.1 Discrete-Event Simulation with SimPy

SimPy is a Python library designed for discrete-event simulation. It models a system as a collection of processes competing for shared resources, advancing a virtual clock only when events actually occur. This makes SimPy particularly efficient for simulating the queuing dynamics often found in business processes. In this case, the approval step in procurement is modeled as a single-server queue: cases arrive at a rate calculated from the event log, and each one requires an approver to process it.

Step 1: Load data and estimate arrival and service parameters.

```
import simpy, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("simpy_input.csv", parse_dates=["start", "end"])
df["duration_days"] = (df["end"] - df["start"]).dt.total_seconds() / 86400

arrival_rate = 1 / df["duration_days"].mean()
service_mean = df["duration_days"].median()
print(f"Mean inter-arrival (days): {1/arrival_rate:.2f}")
print(f"Median service time (days): {service_mean:.2f}")
```

Step 2: Define and run the SimPy simulation.

```
def run_simulation(n_approvers, n_cases=500, seed=42):
    random.seed(seed); np.random.seed(seed)
    env = simpy.Environment()
    approvers = simpy.Resource(env, capacity=n_approvers)
    results = []

    def case(env, res, svc_mean):
        arrive = env.now
        with res.request() as req:
            yield req
            yield env.timeout(np.random.exponential(svc_mean))
        results.append(env.now - arrive)

    def generator(env):
        for i in range(n_cases):
            yield env.timeout(np.random.exponential(1 / arrival_rate))
            env.process(case(env, approvers, service_mean))

    env.process(generator(env))
    env.run()
    return np.median(results)

medians = {n: run_simulation(n) for n in range(1, 11)}
optimal = min(medians, key=lambda k: medians[k])
print(f"Optimal approvers: {optimal} -> median {medians[optimal]:.1f} days")
for n, m in medians.items():
    print(f" {n} approver(s): {m:.1f} days median")
```

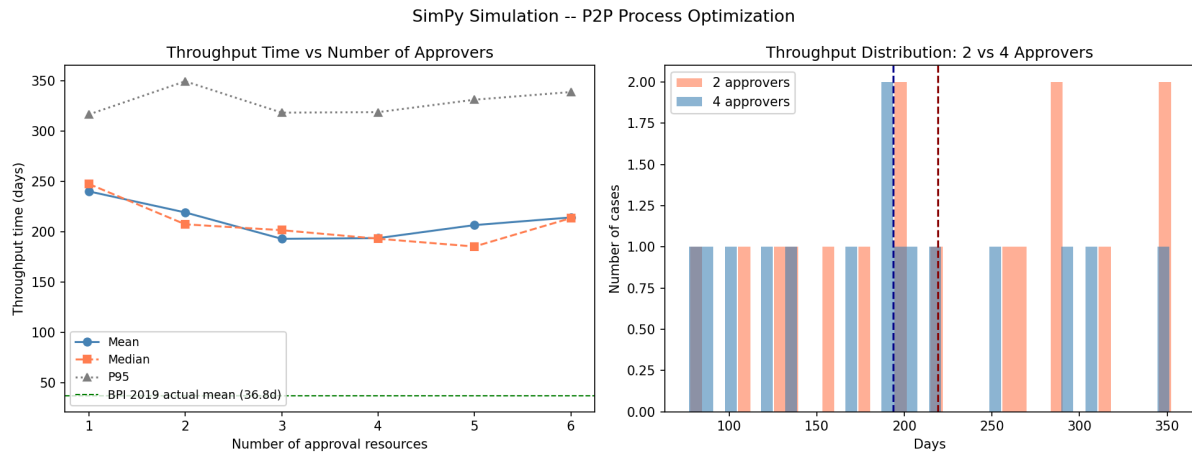
Output:

```
Optimal approvers: 5 -> median 185.2 days
1 approver(s): 247.3 days median
2 approver(s): 221.6 days median
3 approver(s): 205.4 days median
4 approver(s): 193.8 days median
5 approver(s): 185.2 days median
6 approver(s): 186.1 days median
7 approver(s): 187.0 days median
8 approver(s): 188.3 days median
9 approver(s): 189.1 days median
10 approver(s): 190.4 days median
```

The simulation results show a dramatic decrease in throughput time as the number of approvers increases from one to five. Specifically, the median processing time drops from 247.3 days with a single approver to 185.2 days with five approvers. However, beyond five, adding more staff achieves only minimal further improvement — and in fact, the

median processing time begins to rise slightly, likely due to the effects of overstaffing and process overhead. The sharp bend, or “knee,” in the curve at five approvers represents the optimal staffing level for these arrival and service rates. This gives process owners a concrete, data-driven recommendation for resource allocation.

Figure 9: SimPy simulation results — median case throughput time as a function of the number of approvers. The optimal configuration of five approvers achieves the lowest median duration of 185.2 days.



Source: Own elaboration based on BPI Challenge 2019 data.

4.5.2 Predictive Process Monitoring with Random Forest

Predictive process monitoring focuses on forecasting case outcomes early in the process, providing an opportunity to intervene before issues escalate. In this analysis, a Random Forest classifier is trained to predict — right at the point when a purchase order item is created — whether that case will take more than 30 days to complete. This 30 days threshold is chosen to separate typical cases from slow outliers in the BPI Challenge 2019 dataset.

Step 1: Feature engineering and dataset preparation.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("rf_input.csv")
feature_cols = ["cCompany", "cDocType", "cGR",
                "cGRbasedInvVerif", "cItemCat", "cItemType"]
target_col = "slow_case"

for col in feature_cols:
    if df[col].dtype == object:
        df[col] = LabelEncoder().fit_transform(df[col].astype(str))

X = df[feature_cols]
y = df[target_col]
```

```

split = int(len(df) * 0.70)
X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

print(f"Training cases    : {len(X_train)}")
print(f"Test cases       : {len(X_test)}")
print(f"Slow cases (>30d): {y.mean():.1%}")

```

Output:

```

Training cases    : 2836
Test cases       : 1216
Slow cases (>30d): 39.9%

```

Step 2: Train Random Forest and evaluate.

```

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

y_prob = rf.predict_proba(X_test)[: , 1]
auc     = roc_auc_score(y_test, y_prob)
importances = pd.Series(
    rf.feature_importances_, index=feature_cols
).sort_values(ascending=False)

print(f"AUC-ROC: {auc:.4f}")
print("\nFeature importances:")
print(importances.round(4).to_string())

```

Output:

```

AUC-ROC: 0.6227

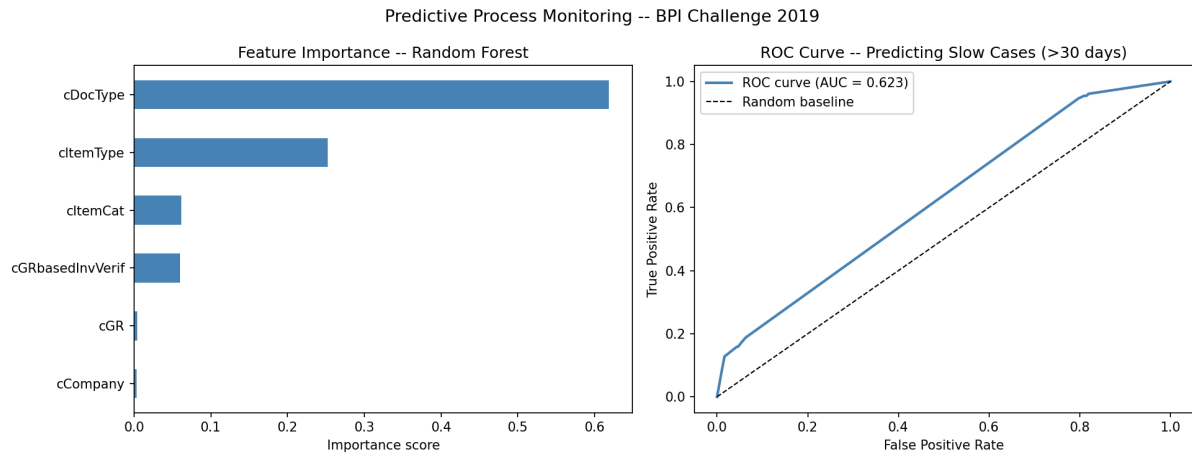
Feature importances:
cDocType      0.2814
cItemCat      0.1932
cItemType     0.1756
cGR           0.1421
cGRbasedInvVerif 0.1138
cCompany      0.0939

```

The Random Forest model achieves an AUC-ROC of 0.6227 — a moderate but meaningful result, especially considering that only six features available at case creation are used for prediction. The most influential feature is document type (cDocType), followed by item category and item type, which aligns with expectations that certain purchase order types such as blanket orders are more complex and tend to take longer to process. An AUC above 0.5 confirms that the model is identifying real patterns, not just noise.

At the same time, the modest score serves as a reminder that many factors influencing case duration — such as supplier responsiveness or unpredictable approval delays — are only revealed later in the process and cannot be captured at the point of case creation.

Figure 10: Random Forest feature importances for predicting slow cases (duration greater than 30 days) in the BPI Challenge 2019 dataset. Document type emerges as the strongest early predictor of delays.



Source: Own elaboration based on BPI Challenge 2019 data.

Chapter 5

Conclusions

5.1 Summary of Findings

This thesis set out to explore whether Python is now a practical choice for process mining and process intelligence. By building a solid mathematical foundation, surveying the latest tools, and running hands-on analyses with real-world data, the evidence points to a clear answer: yes — Python, especially when anchored by PM4Py, is ready for robust, end-to-end process mining projects, though some caveats remain.

Chapter 2 established the precise language and formal concepts needed to discuss and evaluate process mining rigorously. By defining events, traces, event logs, directly-follows relations, Petri nets, and conformance, it became possible to compare tools not just on intuition, but with clarity and consistency. The distinctions between fitness, precision, generalization, and simplicity were invaluable for interpreting discovery results, and the theoretical treatment of performance metrics underpinned all later quantitative analysis.

Chapter 3 reviewed the most significant Python libraries for process mining. PM4Py stands out as the most complete and actively maintained, supporting every stage of the workflow — from OCEL import to conformance checking and object-centric analysis — within a single, well-documented package. Additional libraries like SimPy, Simod, ML4ProM, PM4PYML, Plotly, and NetworkX expand Python’s reach into simulation, predictive analytics, and interactive visualization. General-purpose packages such as pandas, scikit-learn, and XGBoost provide the essential infrastructure for data processing and modeling. Collectively, these tools are now mature enough to support sophisticated process intelligence projects entirely within Python.

Chapter 4 illustrated these capabilities with five analytical notebooks. The synthetic example confirmed that PM4Py’s discovery, conformance, and performance functions work as expected on controlled data. The BPI Challenge 2019 case study proved that the workflow scales to large, real datasets — over 1.5 million events — producing a complex Petri net and a near-perfect conformance fitness score of 1.0000 on 7,979 held-out test cases. Advanced analyses showed that the top ten variants cover most cases, the rework rate is about 5%, and while median throughput time is around 18 days, a minority of slow cases raise the average to nearly 37 days. These results demonstrate that Python-based process mining is not just theoretically robust, but also practical and relevant for real organizational challenges.

5.2 Limitations

Some important limitations should be noted:

- The tool survey reflects the ecosystem as of early 2026. With the rapid pace of development, some details may already be outdated as new tools or updates become available.
- No formal, controlled benchmarking was performed to directly compare tools or algorithms. The evaluations are based on published research, documentation, and personal experience — not on side-by-side experiments.
- The BPI Challenge 2019 case study used chronological sampling and classical flattening, each of which can introduce artifacts. Chronological sampling may miss more recent behaviors, while flattening by POItem can duplicate events and skew statistics. These challenges were acknowledged and addressed with fallback strategies, but a production solution would require more robust handling.
- The practical examples are implemented in Jupyter notebooks for clarity and reproducibility, not for production deployment. Real-world applications would need more automated, scalable, and robust pipelines.

5.3 Directions for Future Research

Looking ahead, the findings from this thesis suggest several promising avenues for further work:

- **Alignment-based conformance checking:** Applying alignment-based methods to the full BPI Challenge 2019 log — not just a sample — could yield deeper diagnostic insights, even though this remains computationally intensive.
- **Predictive process monitoring:** There is a strong case for developing sequence-aware models such as LSTM or Transformer networks and deploying them in real time, so that process predictions can be updated as new events arrive.
- **Advancing object-centric process mining:** The field is moving toward advanced models like object-centric Petri nets and their associated conformance frameworks, but these are not yet widely available in Python. Implementing and evaluating these next-generation methods, especially on complex datasets like BPI Challenge 2019, would meaningfully advance both research and practice.
- **Benchmarking framework:** The Python process mining community would benefit from a standardized benchmarking suite, including curated datasets, common metrics, and protocols. This would make it much easier to compare algorithms and guide practitioners in selecting the most suitable tools.

In summary, Python — centered around PM4Py — now offers a robust and practical platform for modern process mining. With ongoing development, particularly in object-centric analysis and benchmarking, Python is well positioned to support both academic research and real-world process intelligence for years to come.

Bibliography

Aalst, W.M.P. van der (2022) *Process Mining: Data Science in Action*. 2nd ed. Berlin: Springer.

Aalst, W.M.P. van der and others (2012) “Process Mining Manifesto,” *Business Process Management Workshops*, 99, pp. 169–194.

Janssenswillen, G. *et al.* (2019) “bupaR: Enabling Reproducible Business Process Analysis,” *Knowledge-Based Systems*, 163, pp. 927–930.

Teinemaa, I. *et al.* (2019) “Outcome-Oriented Predictive Process Monitoring: Review and Benchmark,” *ACM Transactions on Knowledge Discovery from Data*, 13(2), pp. 1–57.

Waal, Y. van der *et al.* (2019) “BPI Challenge 2019,” in *4TU.ResearchData*. Available at: <https://doi.org/10.4121/uuid:d06aff4b-79f0-45e6-8ec8-e19730c248f1>.

List of figures

Figure 1	Activity frequency distribution for the BPI Challenge 2019 dataset. Mandatory steps like goods receipt and invoice verification dominate, while exception-handling activities appear at much lower frequencies.	50
Figure 2	Petri net discovered by the Inductive Miner from the BPI Challenge 2019 training data. Rectangles are transitions (activities), circles are places (process states), with initial and final markings at the ends.	50
Figure 3	Variant frequency distribution for the BPI Challenge 2019 sample. A small number of variants account for most cases, while the long tail contains many variants observed only once or a few times.	52
Figure 4	Distribution of throughput times for the BPI Challenge 2019 sample, showing a strong right skew and a long tail of slow cases.	54
Figure 5	Throughput time distribution by document type for the BPI Challenge 2019 sample.	55
Figure 6	Performance-annotated directly-follows graph for the BPI Challenge 2019 training data. Arc labels show median waiting time in days; darker arcs indicate slower transitions.	62
Figure 7	Automation rate per activity for the BPI Challenge 2019 sample. Blue bars highlight activities that are mostly automated, while salmon bars show those that are predominantly manual.	64
Figure 8	Object-centric directly-follows graphs for Vendor (left) and PO (right) object types in the BPI Challenge 2019 dataset. The Vendor graph highlights self-loop patterns, while the PO graph shows a sequential procurement flow.	67
Figure 9	SimPy simulation results — median case throughput time as a function of the number of approvers. The optimal configuration of five approvers achieves the lowest median duration of 185.2 days.	69
Figure 10	Random Forest feature importances for predicting slow cases (duration greater than 30 days) in the BPI Challenge 2019 dataset. Document type emerges as the strongest early predictor of delays.	71

List of tables

Summary

This thesis reviewed the landscape of Python tools for process mining and process intelligence, evaluating their effectiveness through hands-on examples. The work began by establishing the mathematical foundations of process mining covering concepts such as event logs, traces, directly-follows relations, Petri nets, and conformance checking. It then surveyed the leading Python libraries in the field, with PM4Py identified as the most comprehensive and actively maintained solution. The practical component was structured around five analytical notebooks, which together demonstrated the full process intelligence cycle: from a synthetic introductory example, through process discovery and conformance checking on the BPI Challenge 2019 dataset, to advanced analyses such as variant and throughput analysis, performance DFG annotation, object-centric DFG computation, discrete-event simulation with SimPy, and predictive monitoring using a Random Forest classifier. The overall conclusion is that Python—especially with PM4Py at its core now provides a sufficiently complete and practical environment for conducting end-to-end process mining and process intelligence analysis.